

QR-HINT: Actionable Hints Towards Correcting Wrong SQL Queries

YIHAO HU, Duke University, USA

AMIR GILAD, Hebrew University, Israel

KRISTIN STEPHENS-MARTINEZ, Duke University, USA

SUDEEPA ROY, Duke University, USA

JUN YANG, Duke University, USA

We describe a system called QR-HINT that, given a (correct) target query Q^* and a (wrong) working query Q , both expressed in SQL, provides actionable hints for the user to fix the working query so that it becomes semantically equivalent to the target. It is particularly useful in an educational setting, where novices can receive help from QR-HINT without requiring extensive personal tutoring. Since there are many different ways to write a correct query, we do not want to base our hints completely on how Q^* is written; instead, starting with the user's own working query, QR-HINT purposefully guides the user through a sequence of steps that provably lead to a correct query, which will be equivalent to Q^* but may still “look” quite different from it. Ideally, we would like QR-HINT's hints to lead to the “smallest” possible corrections to Q . However, optimality is not always achievable in this case due to some foundational hurdles such as the undecidability of SQL query equivalence and the complexity of logic minimization. Nonetheless, by carefully decomposing and formulating the problems and developing principled solutions, we are able to provide provably correct and locally optimal hints through QR-HINT. We show the effectiveness of QR-HINT through quality and performance experiments as well as a user study in an educational setting.

CCS Concepts: • **Information systems** → **Structured Query Language**; • **Applied computing** → **Education**; • **Theory of computation** → **Logic and databases**; **Logic and verification**; **Automated reasoning**.

Additional Key Words and Phrases: SQL Query Equivalence; Boolean Formula Minimization; SQL Debugging

ACM Reference Format:

Yihao Hu, Amir Gilad, Kristin Stephens-Martinez, Sudeepa Roy, and Jun Yang. 2024. QR-HINT: Actionable Hints Towards Correcting Wrong SQL Queries. *Proc. ACM Manag. Data* 2, 3 (SIGMOD), Article 164 (June 2024), 27 pages. <https://doi.org/10.1145/3654995>

1 INTRODUCTION

In an era of widespread database usage, SQL remains a fundamental skill for those working with data. Yet, SQL's rich features and declarative nature can make it challenging to learn and understand. When students encounter difficulties in debugging their SQL queries, they often turn to instructors and teaching assistants for guidance. However, this one-on-one approach is limited in scalability. Syntax errors are easy to fix, but many queries contain subtle semantic errors that may require careful and time-consuming debugging. To save time, the teaching staff is often tempted to give hints based on how the reference solution query is written, ignoring what students have written themselves, but doing so misses opportunities for learning. A SQL query can be written in many

Authors' addresses: Yihao Hu, yihao.hu@duke.edu, Duke University, USA; Amir Gilad, amirg@mail.huji.ac.il, Hebrew University, Israel; Kristin Stephens-Martinez, ksm@cs.duke.edu, Duke University, USA; Sudeepa Roy, sudeepa@cs.duke.edu, Duke University, USA; Jun Yang, junyang@cs.duke.edu, Duke University, USA.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).

ACM 2836-6573/2024/6-ART164

<https://doi.org/10.1145/3654995>

ways that are different in syntax but nonetheless equivalent semantically. Seasoned teaching staff knows how to guide students through a sequence of steps that, starting with their own queries, lead them to a corrected version that is equivalent to the solution query but without revealing the solution query. Our goal is to build a system to help provide this service to students in a more scalable manner.

EXAMPLE 1. Consider the following database (keys are underlined) about beer drinkers and bars: Likes(drinker, beer), Frequents(drinker, bar), Serves(bar, beer, price). Suppose we want to write an SQL query for the following problem: For each beer b that Amy likes and each bar r frequented by Amy that serves b , show the rank of r among all bars serving b according to price (e.g., if r serves b at the highest price, r 's rank should be 1). We assume that there are no ties.

The reference solution query Q^* is given as follows:

```
|| SELECT L.beer, S1.bar, COUNT(*)
|| FROM Likes L, Frequents F, Serves S1, Serves S2
|| WHERE L.drinker = F.drinker AND F.bar = S1.bar
||      AND L.beer = S1.beer AND S1.beer = S2.beer
||      AND S1.price <= S2.price
|| GROUP BY F.drinker, L.beer, S1.bar
|| HAVING F.drinker = 'Amy';
```

Now consider a wrong student query Q :

```
|| SELECT s2.beer, s2.bar, COUNT(*)
|| FROM Likes, Serves s1, Serves s2
|| WHERE drinker = 'Amy'
||      AND Likes.beer = s1.beer AND Likes.beer = s2.beer
||      AND s1.price > s2.price
|| GROUP BY s2.beer, s2.bar;
```

Suggesting good hints to help students fix Q is not easy. First, there are many ways to write a query that is equivalent to Q^* , and queries that look very different syntactically might be semantically similar or equivalent, so relying solely on the syntactic difference between Q and Q^* to propose fixes is ineffective and potentially misleading. In Example 1, even though Q^* has a HAVING clause, it would be confusing to suggest add HAVING to Q , because the condition $\text{drinker} = \text{'Amy'}$ in Q 's WHERE serves the same purpose. Also, even though Q has $\text{Likes.beer} = \text{s2.beer}$ in WHERE while Q^* has $\text{S1.beer} = \text{S2.beer}$, the difference is non-consequential because of the transitivity of equality. Yet another example is $\text{s1.price} > \text{s2.price}$ in Q versus $\text{S1.price} \leq \text{S2.price}$ in Q^* . It would be wrong to suggest changing $>$ to $\leq$ in Q , because an examination of the entire Q would reveal that the student intends s2 (and s1) in Q to serve the role of S1 (and S2) in Q^* . The correct fix is actually changing $>$ to $\geq$.¹

Second, it is often impossible to declare a part of Q as “wrong” since one could instead fix the remainder of Q to compensate for it. For example, we could argue that $\text{s1.price} > \text{s2.price}$ in Q is “wrong,” but there exists a correct query containing precisely this condition, e.g., with $(\text{s1.price} > \text{s2.price OR s1.price} = \text{s2.price})$. Hence, it is difficult to formally define what “wrong” means. Instead of basing our approach heuristically on calling out “wrong” parts, we formulate the problem as finding the “smallest repairs” to Q that make it correct.

Third, hints are for human users, so for a query with multiple issues—which is often the case in practice—we must be aware of the cognitive burden on users and not overwhelm them by asking

¹Another wrong hint would be to suggest changing $\text{COUNT}(\ast)$ to $\text{COUNT}(\ast)+1$ in Q 's SELECT instead of changing the inequality because doing so misses the top-ranked bars. QR-HINT will not make such a mistake.

them to make multiple fixes simultaneously. This desideratum introduces the challenge of planning the sequence of hints and defining appropriate intermediate goals.

Finally, effective hinting faces several fundamental barriers. Realistically, we cannot hope to always provide “optimal” hints because doing so entails solving the query equivalence problem for SQL, which is undecidable [1, 28, 41, 52]; even for decidable query fragments, Boolean expression minimization is known to be on the second level of the polynomial hierarchy (precisely Σ_2^P [16]).

To address the challenges, we propose QR-HINT, a system that, given a target query Q^* and a working query Q , follows the logical execution flow (i.e., FROM→WHERE→GROUPBY→HAVING→SELECT) and produces step-by-step hints for the user to edit the working query to eventually achieve Q^* . The sequence of steps is guaranteed to lead the user on a correct path to eventual correctness. The following example shows QR-HINT helps fix the query in Example 1.

EXAMPLE 2. *Continuing with Example 1, QR-HINT automatically generates the sequence of hints below. Currently built for the teaching staff, QR-HINT only generates the “repairs” below; using these repairs, the teaching staff would then hint to the user in natural language. With the recent advances in generative AI chatbots, it would not be difficult to automate the natural language hints as well; the advantage of using QR-HINT in that setting would be to provide provable guarantees on the quality of hints, which otherwise would be difficult, if not impossible, for generative AI to achieve by itself.*

Stage	QR-HINT repair	Hint in natural language
FROM	<i>Frequents needed</i>	It looks like you are missing one table—read the problem carefully and see what other piece of information you need.
WHERE	$s1.price > s2.price$ $s1.price \geq s2.price$	$\mapsto$ Your WHERE has a small problem with $s1.price > s2.price$. Think through some concrete examples and see how you may fix it.

Note the sequential nature of the hints above; the working query constantly evolves. QR-HINT first focuses on FROM and will only proceed to WHERE after FROM is “viable.” After adding Frequents to FROM, the user will also need to add appropriate join conditions in WHERE; if these were not added correctly, the second step above would suggest additional repairs. It turns out that for this example, only the above two hints are needed to fix the query. In particular, QR-HINT knows not to suggest spurious hints such as adding to Frequents.drinker to GROUP BY or changing s2.beer to Likes.beer in SELECT.

We make the following contributions:

- We develop a novel framework that allows QR-HINT to provide step-by-step hints to fix a working SQL query with the goal of making it equivalent to a target query. This framework formalizes the notion of “correctness” for a sequence of hints, allowing QR-HINT to guarantee that every hint is actionable and is on the right path to achieve eventual correctness. Further, by formulating the hinting problem in terms of finding repair sites in Q with viable fixes, we are able to quantify the quality of the hints.
- Since the optimality of hints, in general, is impossible to achieve due to the foundational hurdles discussed earlier, we aim to provide guarantees on the “local” optimality of QR-HINT in each step. We design practical algorithms with sensible trade-offs between optimality and efficiency.
- We evaluate the performance and efficacy of QR-HINT experimentally. We further perform a user study involving students from current/past database courses offered at the authors’ institution. Our findings indicate that QR-HINT finds repairs that are optimal or close to optimal in practice under reasonable time, and they lead to hints that are helpful for students.

2 RELATED WORK

Debugging Query Semantics. There are two main lines of work toward debugging query semantics (as opposed to syntax or performance). The first line helps debug a query but without knowing the correct (reference) query; in this regard, it differs fundamentally from QR-HINT. Qex [53] is a

tool for generating input relations and parameter values for unit-testing parameterized SQL queries. SQLLint [10–13, 30] detects suspected semantic errors in a query, alerting users to what may be indicative of efficiency, logical, or runtime errors. The work highlights a list of common semantic errors made by students and SQL users [12], but it does not suggest edits, and fixing the suspected errors will not guarantee that the query is correct. Habitat [25, 31] is a query execution visualizer that allows users to highlight parts of a query and view their intermediate results. While it helps users spot possible errors, it gives no edit suggestions if errors exist. More recently, QueryVis [42] turns queries into intuitive diagrams, helping users better understand the semantics of the queries and spot potential errors.

The second line of work, more directly related to QR-HINT, focuses on checking a query against a reference query and/or helping to explain their difference. However, previous work has not been able to suggest small fixes that will make the user query equivalent to the reference query. XData [19] checks the correctness of a query by running the query on self-generated testing datasets based on a set of pre-defined common errors, but it provides no guarantees beyond this pre-defined set. Cosette [21–23] uses constraint solvers and theorem provers to establish the equivalence of two queries or construct arbitrary instances that differentiate them. From a large database instance, RAtest [44] utilizes data provenance to generate a small, illustrative instance to differentiate queries. C-instances [29] aims at constructing small abstract instances based on c-tables [37] that can differentiate two given queries in all possible ways. While Cosette, RAtest, and c-instances can provide examples illustrating how two queries are semantically different, they can only indirectly help users pinpoint errors in the original query; none of them is able to suggest fixes. Chandra et al. [18] developed a grading system that canonicalizes queries by applying rewrite rules and then decides partial credits based on a tree-edit distance between logical plans. However, as query syntax differs significantly from canonicalized plans after rewrite, edits on a canonicalized plan do not translate naturally to small fixes on the original query, making it hard for users to use these edits as hints. Finally, SQLRepair [47] fixes simple errors in an SPJ query using constraint solvers to synthesize/remove WHERE conditions until the query produces correct outputs over all testing instances. Its scope of error is much narrower than what we consider, and its tests-driven nature offers no guarantee of query equivalence.

Program Repair and Feedback for GPL. Several approaches have been developed for program repair in general-purpose programming languages (GPL), but none can be directly applied or easily transferred to SQL. First, a wrong program is usually aligned with reference program(s) ([3, 32, 54]) and fixes are generated based on the selected reference program using various techniques. Such an approach is similar to QR-HINT, but SQL is essentially different from GPL as SQL is declarative and GPLs are usually procedural. While it is possible to write programs in GPL to simulate the execution of a specific SQL query, there is no well-defined mapping between the syntax of SQL and any GPL. As a result, it is impossible to apply such program repair techniques to SQL in general. Another approach is to leverage test cases to synthesize “patches” for the wrong program so that it returns the same output as the reference program for all test cases ([36, 45, 48, 51, 56]). However, such an approach heavily relies on the test cases to cover all possible errors and thus usually fails to guarantee semantic equivalence. Besides the traditional approaches, recent work explores ML algorithms to provide feedback and correction ([8, 9, 20, 33, 34, 43, 46]). In addition, large language models such as GPT-3 [15] have shown an ability to explain the semantics of SQL queries, but does not guarantee the correctness of fixes.

Testing query equivalence. While the query equivalence problem in general is undecidable [1, 5, 50, 52], tools and algorithms are developed to check the equivalence of various classes of queries with restrictions and assumptions [4, 17, 21–23, 26, 38–40, 49, 55, 57]. Although they give a

deterministic answer on equivalence, these tools/algorithms cannot provide any explanation on which parts of the users' queries cause semantic differences from the reference queries.

3 THE QR-HINT FRAMEWORK

Queries. We consider SQL queries that are select-project-join queries with an optional single level of grouping and aggregation. For simplicity of presentation, we assume these are single-block SQL queries with SELECT, FROM (without JOIN operators), and WHERE (with condition defaulting to TRUE if missing) clauses,² together with optional GROUP BY and HAVING clauses. We refer to such a query as an *SPJA* query if it contains grouping or aggregation or DISTINCT; otherwise, we will call it an *SPJ* query.

We assume the default bag (multiset) semantics of SQL. Given query Q , let $F(Q)$ denote the cross product of Q 's FROM tables (including multiple occurrences of the same table, if any); and let $FW(Q)$ denote the query that further filters $F(Q)$ by Q 's WHERE condition (i.e., $FW(Q)$ is a SELECT * query with the same FROM and WHERE clauses as Q). Furthermore, if Q is SPJA, let $FWG(Q)$ denote the (non-relational) query³ that further groups the result rows of $FW(Q)$ according to Q 's GROUP BY expressions (or $\emptyset$ if there are none but Q contains aggregation nonetheless, in which case all result rows belong to a single group). Finally, if Q is SPJA, let $FWGH(Q)$ denote the (non-relational) query that filters the groups of $FWG(Q)$ according to Q 's HAVING conditions (which defaults to TRUE if missing). When discussing equivalence (denoted $\equiv$) among above queries, we require that they return the same bag of result rows (ignoring row and column ordering) for any underlying database instance, and additionally, for queries returning groups, they return the same partitioning of result rows (ignoring group ordering).

SMT Solvers. As with previous work [21, 44, 57], we leverage *satisfiability modulo theory* (SMT) solvers to implement various primitives used by our system. Such a solver can decide whether a formula, modulo the theories it references, is satisfiable, unsatisfiable, or unknown (beyond the solver's capabilities). Specifically, we use the popular SMT solver Z3 [24] to implement the following three primitives. Given two quantifier-free expressions, $\text{IsEquiv}(e_1, e_2)$ tests whether $e_1 \Leftrightarrow e_2$ (for logic formulae such as those in WHERE) or $e_1 = e_2$ (for value expressions such as those in SELECT or GROUP BY). Given a logic formula p , $\text{IsUnsatisfiable}(p)$ and $\text{IsSatisfiable}(p)$ return, respectively, whether p is satisfiable or unsatisfiable, respectively. All above primitives may return "unknown" when Z3 is unsure about its answer. However, when they return true, Z3 guarantees that the answer is not a false positive. Our algorithms in subsequent sections act only on (true) positive answers from these primitives. For complex uses, it is often convenient to frame equivalence/satisfiability testing using a *context* C , or a set of logical assertions (e.g., types declaration, known constraints, and inference rules) under which testing is done. We use subscripts to specify the context: e.g., $\text{IsUnsatisfiable}_C(p)$ is a shorthand for $\text{IsUnsatisfiable}((\bigwedge_{c \in C} c) \wedge p)$.

EXAMPLE 3. Consider a query with a WHERE condition stipulating that $A > 100$ for an INT-typed column A , as well as a HAVING condition $\text{MAX}(A) \geq 101$. We might wonder whether the HAVING condition is unnecessary. To this end, we call $\text{IsUnsatisfiable}_C(p)$ with

$$C : \left\{ \begin{array}{l} A \text{ has type } \text{Array}(\mathbb{Z}) \\ \forall i \in \mathbb{N} : A[i] > 100 \\ \text{MAX has type } \text{Array}(\mathbb{Z}) \rightarrow \mathbb{Z} \\ \forall i \in \mathbb{N}, X \text{ of type } \text{Array}(\mathbb{Z}) : \text{MAX}(X) \geq X[i] \end{array} \right\}, \quad p : \neg(\text{MAX}(A) \geq 101).$$

²We can handle a query with common table expressions (WITH) and subqueries in FROM that are aggregation-free, as well as non-outer JOINS in FROM, by rewriting the query into single-block SQL.

³This query is non-relational because it returns, besides the bag of rows from $FW(Q)$, a partitioning of them into groups.

The first two assertions in C are derived from the type of A and the WHERE conditions; here the array-typed A refers to a collection of A values. The last two specify (some) general inference rules on the SQL aggregate function MAX. Z3 correctly returns true, meaning that $\text{MAX}(A) \geq 101$ must be true under C and is therefore unnecessary.

Our use of Z3 for reasoning with SQL aggregation, such as the example above, goes beyond the practice in previous work, where aggregation functions are mostly treated as uninterpreted functions. For example, to test the equality of two aggregates, [57] conservatively checks whether input value sets or multisets for the aggregate function are equal. In contrast, we encode properties of SQL aggregation functions in a way that allows Z3 to reason with them. As formulae become more complicated, e.g., with quantifiers and arrays, Z3 no longer offers a complete decision procedure (as there exists no decision procedure for first-order logic) and may return “unknown” more often. Nonetheless, practical heuristics employed by Z3 allow it to handle many cases of practical uses to QR-HINT.

3.1 Approach

Given a (*syntactically correct*) working query Q and a target query Q^* , QR-HINT provides hints in *stages* to help the user edit the working query incrementally until it becomes *semantically equivalent* to Q^* . Each stage focuses on one specific syntactic fragment of the working query. QR-HINT gives actionable hints for the user to edit this fragment with the aim of bringing Q a step “closer” to being equivalent to Q^* . QR-HINT strives to suggest the smallest edits possible and avoid suggesting unnecessary edits. Upon passing a *viability check*, the working query Q clears the current stage and moves on to the next. After clearing all stages, QR-HINT guarantees that $Q \equiv Q^*$ (even if syntactically they are still different).

We now briefly outline the concrete stages of QR-HINT; the details will be presented in the subsequent sections.

For an SPJ query, there are three stages. (1) We start with Q ’s FROM clause (Section 4) and make sure that its list of tables can eventually lead to a correct query; following this stage, $F(Q) \equiv F(Q^*)$. (2) Next, we provide hints to repair Q ’s WHERE clause (Section 5) such that $\text{FW}(Q) \equiv \text{FW}(Q^*)$, i.e., the repaired query returns the same sub-multiset of rows as Q^* that satisfy the WHERE clause, ignoring SELECT. (3) Finally, we handle Q ’s SELECT clause and ensure the working query returns correct output column values. Importantly, we make inferences of equivalence under the premise that all rows before SELECT already satisfy WHERE; this use of WHERE allows us to infer more equivalent cases and avoid spurious hints.

For an SPJA query, there are five stages. (1) The *first stage* handles FROM as in the SPJ case. (2) The *second stage* handles WHERE, but with a twist. As we have seen from Example 1, some condition can be either WHERE or HAVING, and it would be misleading to hint its absence from WHERE to be wrong; hence, QR-HINT will look “ahead” at the two queries’ HAVING and GROUP BY clauses to avoid misleading the user. At the end of this stage, instead of insisting that $\text{FW}(Q) \equiv \text{FW}(Q^*)$ for the original Q^* , we may rewrite Q^* (by legally moving some conditions between WHERE and HAVING) as needed first. (3) The *third stage* is GROUP BY, where we provide hints to edit Q ’s GROUP BY expressions to achieve equivalent grouping, i.e., $\text{FWG}(Q) \equiv \text{FWG}(Q^*)$. Here, we infer equivalence under the premise that the rows to be grouped all satisfy WHERE. (4) The *fourth stage* is HAVING, where we provide hints to repair Q ’s HAVING condition in the same vein as WHERE; however, inferences in this stage would additionally consider both WHERE and GROUP BY, and they are more challenging because of aggregation functions. After this stage, we have $\text{FWGH}(Q) \equiv \text{FWGH}(Q^*)$. (5) The *fifth and final stage* is SELECT, which is similar to the SPJ case, but with the challenge of handling aggregation functions while simultaneously considering WHERE, GROUP BY, and HAVING.

Progress and Correctness. Note that to clear a stage, the user only needs to come up with a fix to pass the viability checks up to this stage. Even though QR-HINT may examine the queries in their entirety, the user does not have to think ahead about how to make the entire query correct.⁴ Moreover, once a stage is cleared, QR-HINT never requires the user to come back to fix the same fragment again. This stage-by-stage design with “localized” hints helps limit the cognitive burden on the user.

The following theorem formalizes the intuition that this stage-based approach leads to steady, forward progress toward the goal of fixing the working query. It follows from the observation that our solution for each stage ensures the properties asserted below, which we will show stage by stage in the subsequent sections.

THEOREM 3.1. *Let $Q_0 = Q$ denote the initial working query and Q^* denote the target query. Let V_i denote the viability check for stage i , and Q_i denote the working query upon clearing stage i , where Q_i satisfies $V_1, V_2, \dots, V_i$. We say that two queries are stage- i consistent if they are identical syntactically except in the fragments that stage $i + 1$ and beyond focus on. For each stage i , the following hold:*

(Hint leads to fix) *If Q_{i-1} fails to satisfy V_i , there exists a query $\hat{Q}_i$ such that $\hat{Q}_i$ satisfies $V_1, V_2, \dots, V_i$, $\hat{Q}_i$ is stage- $(i - 1)$ consistent with Q_{i-1} , and $\hat{Q}_i$ follows the stage- i hint provided by QR-HINT.*

(Fix leads to eventual correctness) *There exists a query $\hat{Q}$ such that $\hat{Q} \equiv Q^*$ and $\hat{Q}$ is stage- i consistent with Q_i .*

We delegate all proofs in this paper to the full technical report [35].

Optimality. Ideally, we would like QR-HINT to suggest the “best possible” hints, e.g., those leading to minimum edits to the working query. Unfortunately, it is impossible for any system to provide such a guarantee in general, because doing so entails being able to determine the equivalence of SQL queries: if $Q \equiv Q^*$ to begin with, the system should not suggest any fix. It is well-known that the equivalence of first-order queries with only equality comparisons is undecidable [1]. Under bag semantics, even the decidability of equivalence of conjunctive queries has not been completely resolved [41]. Once we open up to the full power of SQL, which can express integer arithmetic, even equivalence of selection predicates becomes undecidable via a simple reduction to the satisfiability of Diophantine equations [28].

Given the foundational hurdles above, QR-HINT seeks a pragmatic solution. Instead of offering any global guarantee on the optimality of its hints, which is impossible, QR-HINT establishes, for each stage, guarantees on the necessity or minimality of its hints under certain assumptions. For example, for the FROM stage, QR-HINT guarantees its suggested fixes are optimal for SPJ queries, but for some SPJA queries, it may suggest a fix that turns out to be unnecessary. As another example, for the WHERE stage, the optimality of QR-HINT depends on, among other things, Z3-based primitives offering *complete* decision procedures. In each subsequence section, we will state any such assumption explicitly.

Finally, it is important to note that QR-HINT’s progress and correctness properties (Theorem 3.1) do *not* rely on these assumptions. In the worst case, the user may be hinted to make some fixes that are unnecessary or unnecessarily big, but QR-HINT will still ensure that the user gets a correct working query in the end.

Limitations. Following Theorem 3.1, QR-HINT is guaranteed to generate correct hints for select-project-join queries with an optional single level of grouping and aggregation. On the other hand, QR-HINT currently has several limitations. 1) QR-HINT may sometimes suggest suboptimal or even

⁴In some cases, just to maintain syntactic correctness, a fix may necessitate trivial edits to fragments handled in future stages: e.g., if we remove a table from FROM, we will need to remove references to this table in the rest of the query. However, the user never needs to worry about making those edits semantically correct—that responsibility falls on future stages.

unnecessary fixes (even though they still lead to correct queries), as discussed above; the reason lies in fundamental hurdles due to the undecidability of SQL query equivalence and the use of heuristics to tame complexity. 2) QR-HINT currently does not handle NULL values and assumes that all database columns are NOT NULL. With some additional effort and complexity, QR-HINT can be extended to handle NULL using the technique in [57] of encoding each column with a pair of variables in Z3 (one for its value and the other a Boolean representing whether it is NULL). The same applies to OUTER JOIN. 3) Except the case of aggregation-free subqueries in FROM mentioned in Footnote 2, QR-HINT does not support subqueries in general. Subqueries involving aggregation, in general, cannot be folded into the outer query block. Subquery constructs such as NOT EXISTS and NOT IN entail supporting queries involving the difference operator, which we have not yet studied. If we do not care about the number of duplicates in the result, positive subqueries with EXISTS and IN could be rewritten as part of the join in the outer select-project-join query and supported as such. However, this approach is unsatisfactory, especially since our handling of FROM (Section 4) does assume that duplicates matter. In general, more work is needed to develop a comprehensive solution for subqueries. 4) Finally, QR-HINT does not consider database constraints such as keys and foreign keys. While we can, in theory, encode some constraints as logical assertions and include them as part of the context when calling Z3, these assertions (with quantifiers) can significantly hamper Z3's performance. Future work is needed to develop more robust algorithms for incorporating constraints.

4 FROM STAGE

This stage aims to ensure $F(Q) \equiv F(Q^*)$. Recall that a FROM clause may reference a table T multiple times, and each reference is associated with a distinct alias (which defaults to the name of T). Each column reference must resolve to exactly one of these aliases. Let $\text{Tables}(Q)$ denote the multiset of tables in the FROM clause of Q , and let $\text{Aliases}(Q)$ denote the set of aliases they are associated with in Q . With a slight abuse of notation, given table T , let $\text{Aliases}(Q, T)$ denote the subset of $\text{Aliases}(Q)$ associated with T (a non-singleton $\text{Aliases}(Q, T)$ implies a self-join involving T). Given an alias $t \in \text{Aliases}(Q)$, let $\text{Table}(Q, t)$ denote the table that t is associated with in Q .

The viability check (Theorem 3.1, stage 1) for FROM is simple:

$$V_1 : \text{Check if } \text{Tables}(Q) \stackrel{\sqsubseteq}{=} \text{Tables}(Q^*)$$

where $\stackrel{\sqsubseteq}{=}$ denotes multiset equality. If the working query Q fails the viability check, QR-HINT simply hints, for each table T whose counts in $\text{Tables}(Q)$ and $\text{Tables}(Q^*)$ differ (including cases where T is used in one query but not the other), that the user should consider using T more or less to make the counts the same. It is straightforward to see that this hint leads to a fix that makes $\text{Tables}(Q) \stackrel{\sqsubseteq}{=} \text{Tables}(Q^*)$, which enables the user to further edit Q into some $\hat{Q} \equiv Q^*$ without retouching FROM: at the very least, one can make $\hat{Q}$ isomorphic to Q^* up to the substitution of table references with those in $\text{Aliases}(Q)$. This observation establishes the progress and correctness properties (see Theorem 3.1) of FROM-stage hints, which we state below along with the remark that $F(Q) \equiv F(Q^*)$ after this stage.

LEMMA 4.1. *QR-HINT's FROM-stage hint leads to a fixed working query Q_1 that (1) passes the viability check V_1 $\text{Tables}(Q_1) \stackrel{\sqsubseteq}{=} \text{Tables}(Q^*)$; (2) satisfies $F(Q_1) \equiv F(Q^*)$; and (3) leads to eventual correctness.*

While the correctness of the FROM-stage hint is straightforward, its optimality is surprisingly strong. The following lemma states that the viability check is, in fact, necessary—regardless of what could be done in WHERE and SELECT—under reasonable assumptions.

LEMMA 4.2. *Two SPJ queries Q^* and Q cannot be equivalent under bag semantics if $\text{Tables}(Q^*) \not\equiv \text{Tables}(Q)$ assuming no database constraints are present, and there exists some database instance for which either Q^* or Q returns a non-empty result.*⁵

Table Mappings. To facilitate analysis in subsequent stages, QR-HINT needs a way to “unify” table and column references in Q and Q^* so that all of them use the same set of table aliases.

DEFINITION 1. *Given queries Q^* and Q over the same schema where $\text{Tables}(Q^*) \equiv \text{Tables}(Q)$, a table mapping from Q^* to Q is a bijective function $m : \text{Aliases}(Q^*) \rightarrow \text{Aliases}(Q)$ with the property that two corresponding aliases are always associated with the same table, i.e., $\forall t \in \text{Aliases}(Q^*) : \text{Table}(Q^*, t) = \text{Table}(Q, m(t))$.*

If the queries have no self-joins, it is straightforward to establish this mapping by table names. With self-joins, however, it can be tricky because we must match multiple roles played by the same table across queries. The information contained in FROM alone would be insufficient for matching. One approach is to explore every possible table mapping and select the one that leads to the minimum fix. Doing so would blow up complexity by a factor exponential in the number of self-joined tables. QR-HINT instead opts for a heuristic that picks the single most promising table mapping. Here we describe the heuristic briefly. For each alias, we build a “signature” that captures how its columns are used by various parts of the query in a canonical fashion. We define a distance (cost) metric for the signatures. Then, for each table involved in self-joins, to determine the mapping between its aliases in Q and Q^* , we construct a bipartite graph consisting of these aliases and solve the minimum-cost bipartite matching problem. We illustrate the high-level idea using the example below.

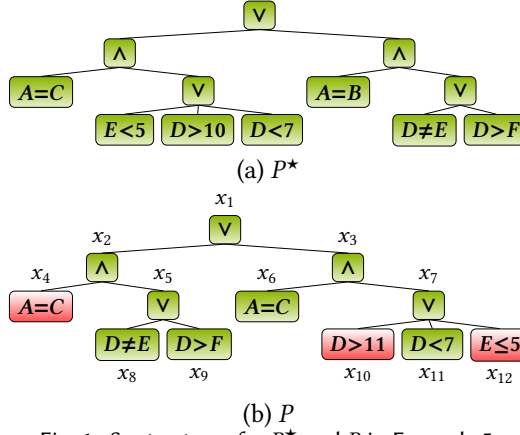
EXAMPLE 4. *Continuing with Example 1, the following are signatures (one per column) for $S1$ and $S2$ in Q^* and $s1$ and $s2$ in Q .*

	$S1$ in Q^*	$S2$ in Q^*	$s1$ in Q	$s2$ in Q
WHERE & bar:	$=\{F.bar\}$	$=\{F.bar\}$	None	None
HAVING beer:	$=\{L.beer, S2.beer\}$	$=\{L.beer, S2.beer\}$	$=\{Likes.beer, s2.beer\}$	$=\{Likes.beer, s2.beer\}$
price:	$\leq \{S2.price\}$	$\geq \{S2.price\}$	$> \{s2.price\}$	$< \{s1.price\}$
GROUP BY	$\{bar, beer\}$	$\{beer\}$	$\{beer\}$	$\{beer\}$
SELECT bar:	$\{2\}$	$\emptyset$	$\emptyset$	$\{2\}$
beer:	$\{1\}$	$\{1\}$	$\{1\}$	$\{1\}$
price:	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

For example, $S1.beer$ ’s WHERE/HAVING signature says that it is involved in an equality comparison with both $L.beer$ and $S2.beer$; the latter is inferred—QR-HINT automatically adds column references and constants that obviously belong to the same equivalence class. Likewise, $S1$ ’s GROUP BY signature includes both bar and $beer$, with the latter added because of its equivalence to the GROUP BY column $L.beer$. When comparing signatures, all aliases are replaced by table names (which is a heuristic simplification); therefore, all four WHERE/HAVING signatures above for $beer$ are considered the same. In this case, what makes the difference in bipartite matching turns out to be the SELECT signatures for bar , which clearly favors the mapping with $S1 \mapsto s2$ and $S2 \mapsto s1$.

Once we have selected the table mapping m , we can then “unify” Q^* and Q . For convenience, we simply rename each alias a in Q^* to $m(a)$; in subsequent sections, we shall assume that Q^* and Q have consistent column references.

⁵The assumption of a not-always-empty result may seem out of the blue, but it is necessary. For example, queries $\text{SELECT } 1 \text{ FROM } R \text{ WHERE FALSE}$ and $\text{SELECT } 1 \text{ FROM } R, R \text{ WHERE FALSE}$ are equivalent—both always return empty results. However, if at least one of Q^* and Q can return non-empty results, $\text{Tables}(Q^*) \equiv \text{Tables}(Q)$ becomes necessary for equivalence. Our proof of Lemma 4.2, in fact, builds on such a non-empty result.

Fig. 1. Syntax trees for P^* and P in Example 5.

5 WHERE STAGE

WHERE is our most involved stage, aimed at making small edits to the WHERE condition of Q so that it becomes logically equivalent to that of Q^* , thereby ensuring $\text{FW}(Q^*) \equiv \text{FW}(Q)$ (recall from Section 3.1). Let P and P^* denote the WHERE predicates in Q and Q^* , respectively. We assume that they have already been unified by the selected table mapping to have the same set of column references, as discussed in Section 4. The viability check for the WHERE stage (Theorem 3.1, stage 2) is simply that P is logically equivalent to P^* :

$$V_2 : \text{Check if } P \Leftrightarrow P^*$$

As discussed in Section 1, if $P \Leftrightarrow P^*$, there are many different ways to modify P so that becomes logically equivalent to P^* , and it is impossible to declare any part of P as definitively “wrong.” Therefore, we suggest the smallest possible edits on P to reduce the cognitive burden on the user. We formalize the notion of “small edits” below. We represent P and P^* using syntax trees, where:

- Internal (non-leaf) nodes represent logical operators $\wedge$, $\vee$, and $\neg$. Let $\text{op}(x)$ denote the operator associated with node x , and $\text{Children}(x)$ denote the x ’s child nodes. If $\text{op}(x)$ is $\neg$, $|\text{Children}(x)| = 1$. If $\text{op}(x) \in \{\wedge, \vee\}$, $|\text{Children}(x)| \geq 2$.
- Leaf nodes are atomic predicates involving column references and/or literals. We treat each unique column reference as a free variable over the domain of the referenced column. We support basic SQL types as well as standard comparison, arithmetic, and string operators to the extent supported by Z3, e.g.: $A > 5$, $B \leq 2C - 10$, $D \text{ LIKE 'Eve\%'}$.

EXAMPLE 5. Consider the following logical formulae P^* and P where A, B, C, D, E are integers:

$$P^*: (A=C \wedge (E < 5 \vee D > 10 \vee D < 7)) \vee (A=B \wedge (D \neq E \vee D > F))$$

$$P: (A=C \wedge (D \neq E \vee D > F)) \vee (A=C \wedge (D > 11 \vee D < 7 \vee E \leq 5))$$

The syntax trees of P^* and P are shown in Figure 1.

DEFINITION 2 (REPAIR FOR SQL PREDICATE). Given a quantifier-free logical formulae P represented as a tree, a repair of P is a pair $(S, \mathcal{F})$ where S is a set of disjoint subtrees of P called the repair sites, and $\mathcal{F}$ is function that maps each site $x \in S$ to a new formulae $\mathcal{F}(x)$ called the fix for x . Given a target predicate P^* , a repair $(S, \mathcal{F})$ for P is correct if applying it to P —i.e., replacing each $x \in S$ with $\mathcal{F}(x)$ —results in a formulae P' such that $P' \Leftrightarrow P^*$.

Algorithm 1: RepairWhere(x, x^*, n)**Input** : a wrong predicate x , a correct predicate x^* , and a cap n on the number of repair sites**Output** : a repair $(\mathcal{S}, \mathcal{F})$ with minimum cost

```

1 let  $\mathcal{S}_o = \emptyset, \mathcal{F}_o = \emptyset$ ;
2 let  $c_o$  denote the minimum cost so far, and  $\infty$  initially;
3 foreach set  $\mathcal{S}$  of  $\leq n$  disjoint subtrees in  $x$ , in ascending  $|\mathcal{S}|$  order do
4   if  $\text{Cost}(\mathcal{S}, \cdot) \geq c_o$  then // cost due to # sites alone is already too big
5     return  $(\mathcal{S}_o, \mathcal{F}_o)$ ; // safe to stop now
6   if  $x^* \in \text{CreateBounds}(x, \mathcal{S})$  then
7     let  $\mathcal{F} = \text{DeriveFixes}(x, \mathcal{S}, x^*, x^*)$ ;
8     if  $c_o > \text{Cost}(\mathcal{S}, \mathcal{F})$  then
9       let  $\mathcal{S}_o = \mathcal{S}, \mathcal{F}_o = \mathcal{F}$ ;
10 return  $(\mathcal{S}_o, \mathcal{F}_o)$ ;

```

DEFINITION 3 (COST OF A REPAIR). Given target predicate P^* , the cost of a repair $(\mathcal{S}, \mathcal{F})$ for P is:

$$\text{Cost}(\mathcal{S}, \mathcal{F}) = w \cdot |\mathcal{S}| + \sum_{s \in \mathcal{S}} \frac{\text{dist}(s, \mathcal{F}(s))}{|P| + |P^*|}, \text{ where} \quad (1)$$

$$\text{dist}(s, \mathcal{F}(s)) = |s| + |\mathcal{F}(s)|, \text{ and} \quad (2)$$

$w \in \mathbb{R}^+$ controls the relative weights of the cost components.

Here, we simply define $\text{dist}(\cdot, \cdot)$ to be the number of nodes deleted and inserted by the repair; other notions of edit distance could be used too. The denominator under $\text{dist}(\cdot, \cdot)$ serves to normalize the measure relative to the sizes of the queries. Also, note that the $w \cdot |\mathcal{S}|$ term adds a fixed penalty for each additional repair site. Intuitively, QR-HINT will present all repair sites (without the associated fixes) to the user as a hint. Even a moderate number of repair sites will pose a significant cognitive challenge—if there were so many issues with P , we might as well ask the user to rethink the whole predicate (which would be a single repair site at the root). In our experiments (Section 9), we set $w = 1/6$, and the number of repair sites per WHERE rarely goes above two or three.

EXAMPLE 6. Consider Figure 1. One correct repair for P consists of three sites (x_4, x_{10}, x_{12}) and the corresponding fixes $(A=B, D>10, E<5)$. The cost for this repair is $3w + \frac{3 \times (1+1)}{12+12} = \frac{1}{2} + \frac{1}{4} = 0.75$.

Another correct repair for P consists of two sites (x_5, x_3) and the corresponding fixes $E<5 \vee D>10 \vee D<7$ and $A=B \wedge (D \neq E \vee D>F)$. The cost for this repair is $2w + \frac{(4+3)+(5+6)}{12+12} = \frac{1}{3} + \frac{3}{4} \approx 1.08$.

A trivial single-site repair that replaces the entire P with P^* would have cost $1w + \frac{(12+12)}{12+12} \approx 1.16$.

Algorithm 1 is our overall procedure for computing a minimum-cost repair for a predicate. It considers all possible sets of repair sites, prioritizing smaller ones because the number of repair sites heavily influences the repair cost, and stopping once the lowest cost found so far is no greater than a conservative lower bound on the cost of the repairs to be considered. In the worst case, the number of repairs to be considered is exponential in the size of P , but in practice, the early stopping condition usually kicks in when the number of repair sites is 2 or 3, so the number of repairs considered is usually quadratic or cubic in $|P|$.

The two key building blocks of Algorithm 1 are CreateBounds and DeriveFixes, which we describe in more detail in the remainder of this section. Intuitively, CreateBounds (Section 5.1) provides a quick and “exact” test to determine whether a given set of repair sites could ever lead to a correct repair. If yes, DeriveFixes (Section 5.2) then finds the “optimal” fixes for these repair sites. Our algorithms use Z3, so their exactness and optimality depend on Z3’s completeness for the types of predicates they are

Algorithm 2: CreateBounds($x, \mathcal{S}$)

Input : a predicate x , and a set $\mathcal{S}$ of disjoint subtrees (repair sites) of x
Output : lower and upper bounds for x achievable by fixing $\mathcal{S}$

```

1 if  $x \in \mathcal{S}$  then return [false, true] ;
2 else if  $x$  is atomic then return [ $x, x$ ] ;
3 else if  $op(x) \in \{\wedge, \vee\}$  then
4   foreach  $c \in Children(x)$  do
5     let [ $l_c, u_c$ ] = CreateBounds( $c, \mathcal{S}[c]$ );6
6   return [ $\bigwedge_{c \in Children(x)} l_c, \bigvee_{c \in Children(x)} u_c$ ] where  $\Theta = op(x)$ ;
7 else //  $op(x)$  is  $\neg$ 
8   let  $c = Children(x)[0]$ ; // the only child of  $x$ 
9   let [ $l_c, u_c$ ] = CreateBounds( $c, \mathcal{S}[c]$ );
10  return [ $\neg u_c, \neg l_c$ ];

```

given. DeriveFixes's optimality further hinges on a Boolean minimization procedure (MinBoolExp) that it also uses. On the other hand, since Z3 inferences are sound, progress and correctness (Section 3.1) are guaranteed.

LEMMA 5.1. WHERE-stage hint leads to a fixed working query Q_2 with WHERE condition that 1) passes the viability check $P \Leftrightarrow P^*$; 2) satisfies $FW(Q_2) \equiv FW(Q^*)$; and 3) leads to eventual correctness.

LEMMA 5.2. Given P and P^* , assuming that Z3 inference is complete with respect to the logic exercised by P and P^* , and that MinBoolExp finds a minimum-size Boolean formula equivalent to its given input, the repair returned by RepairWhere($P, P^*, |P|$) is optimal (i.e., has the lowest possible cost) if there exists an optimal repair that either contains a single site or has all its sites sharing the same parent in P .

Note that Lemma 5.2 provides optimality for two important cases that commonly arise in practice: 1) P makes a single (presumably small) mistake; 2) P is either conjunctive or disjunctive (because all atomic-predicate nodes share the same $\wedge$ or $\vee$ parent node).

5.1 Viability of Repair Sites

The key idea is that, given a set of repair sites in P , we can quickly compute a “bound” that precisely defines what can be accomplished by *any* fixes at these sites (and only at these sites). We first give the definition of bounds and introduce some notations. Give quantifier-free logical formulae $P_\perp$, P , and $P_\top$ such that $P_\perp \Rightarrow P \Rightarrow P_\top$, we say that $[P_\perp, P_\top]$ is a *bound* for P , denoted $P \in [P_\perp, P_\top]$. We call $P_\top$ an *upper bound* of P and $P_\perp$ a *lower bound* of P .

CreateBounds($P, \mathcal{S}$) (Algorithm 2) computes a precise bound for any predicate that can be obtained by fixing P at the given set $\mathcal{S}$ of repair sites. It works by computing a bound for each node in P in a bottom-up fashion, starting from the repair sites or leaves of P . We call these bounds *repair bounds*. Intuitively, the repair bound at a repair site would be [false, true], because a fix can change it to any logical formula. If a subtree contains no repair sites underneath, it would have a very tight repair bound of [p, p], where p denotes the formulae corresponding to the subtree, which is unchangeable by the given repair. The internal logical nodes combine and transform these bounds in expected ways in Algorithm 2.

EXAMPLE 7. Given repair sites $\{x_4, x_{10}, x_{12}\}$ for P in Figure 1, CreateBounds computes the repairs bounds shown below.

⁶For node x in P , $\mathcal{S}[x]$ denotes the subset of $\mathcal{S}$ that belong to the subtree rooted at x .

Node(s)	repair lower bound	repair upper bound
x_4	false	true
x_8, x_9, x_5	original predicate in P	
x_2	false	$D \neq E \vee D > F$
x_6	original predicate in P	
x_{10}	false	true
x_{11}	original predicate in P	
x_{12}	false	true
x_7	$D < 7$	true
x_3	$A = C \wedge D < 7$	$A = C$
$x_1 (P)$	$A = C \wedge D < 7$	$D \neq E \vee D > F \vee A = C$

The following shows that repair bounds computed by CreateBounds are valid. The proof uses an induction on the structure of P .

LEMMA 5.3 (VALIDITY OF REPAIR BOUNDS). *Given a predicate P and a set $\mathcal{S}$ of repair sites, $\text{CreateBounds}(P, \mathcal{S})$ outputs two predicates $P_{\perp}$ and $P_{\top}$, such that applying any repair $(\mathcal{S}, \mathcal{F})$ (with the given $\mathcal{S}$) will result in a predicate $P' \in [P_{\perp}, P_{\top}]$.*

Lemma 5.3 immediately yields a method for deciding whether a candidate set $\mathcal{S}$ of repair sites is viable: if the target formula $P^* \notin [P_{\perp}, P_{\top}]$ given $\mathcal{S}$, then there does not exist a set of correct fixes $\mathcal{F}$ for $\mathcal{S}$. The next natural question to ask is: if the target formula $P^* \in [P_{\perp}, P_{\top}]$ given $\mathcal{S}$, is it always possible to find some correct fixes? The answer to this question is yes—and Section 5.2 will provide constructive proof. Hence, repair bounds provide a *precise* test of whether a set $\mathcal{S}$ of repair sites is viable.

For example, continuing from Example 7, using Z3, it is easy to verify that $P^* \in [A = C \wedge D < 7, D \neq E \vee D > F \vee A = C]$; therefore, $\{x_4, x_{10}, x_{12}\}$ is a viable set of repair sites for P with respect to P^* .

5.2 Derivation of Fixes

Suppose the target formula P^* falls within the repair bound $[P_{\perp}, P_{\top}]$ computed by $\text{CreateBounds}(P, \mathcal{S})$. We now introduce DeriveFixes (Algorithm 3) that computes correct fixes $\mathcal{F}$ for $\mathcal{S}$. The idea is to traverse P 's syntax tree top-down and derive a *target bound* for each node x . As long as we repair subtrees rooted at x 's children such that the resulting predicates fall within their respective target bounds, we will have a repair for x that makes its result predicate fall within x 's target bound. We start from P 's root with the desired target bound $[P^*, P^*]$ and “push it down”; whenever we reach a repair site, its fix would simply be the smallest formula (found by MinFix) that falls within the target bound we have derived for the repair site.

The intuition behind how to “push down” the target bound at node x to its children is as follows. First, the repair bound on a child c of x dictates what repairs are possible—the target bound we set for c must be bound by its repair bound. However, we want to tighten the repair bound as little as possible because a looser target bound gives MinFix more freedom in finding a small formula. As a simple example, consider the target bound $[a_1 \wedge a_2, (a_1 \wedge a_2) \vee a_3]$, where a_1, a_2, a_3 are independent atomic predicates. The smallest formula within this bound is $a_1 \wedge a_2$. However, if the target bound were looser, e.g., $[a_1 \wedge a_2 \wedge a_3, (a_1 \wedge a_2) \vee a_3]$, the smallest formula within this new bound would be just a_3 , smaller than before.

Lines 15–22 of Algorithm 3 spells out our strategy. We will illustrate the key ideas with Example 7 and Figure 1. Consider pushing down the target bound of $[P^*, P^*]$ at x_1 to x_2 and x_3 . Note that our choices of target bounds for x_2 and x_3 are constrained by their respective repair bounds in the table of Example 7; in general, we will need to raise these lower bounds and/or lower these upper bounds

Algorithm 3: $\text{DeriveFixes}(x, \mathcal{S}, l^*, u^*)$

Input : a predicate x , a set $\mathcal{S}$ of disjoint subtrees (repair sites) of x , and a target bound $[l^*, u^*]$ for x to achieve by fixes

Output : a repair represented as a set of (s, f) pairs, one for each $s \in \mathcal{S}$

```

1 if  $x \in \mathcal{S}$  then return  $\{(x, \text{MinFix}(l^*, u^*))\}$  ;
2 else if  $x$  is atomic then return  $\emptyset$  ;
3 else if  $\text{op}(x)$  is  $\neg$  then
4   let  $c = \text{Children}(x)[0]$ ; // the only child of  $x$ 
5   return  $\text{DeriveFixes}(c, \mathcal{S}[c], \neg u_0^*, \neg l_0^*)$ ;
6 let  $\Theta = \text{op}(x)$ ; // either  $\wedge$  or  $\vee$  at this point
7 foreach  $c \in \text{Children}(x)$  do
8   let  $[l_c, u_c] = \text{CreateBounds}(c, \mathcal{S}[c])$ ;
9 let  $\mathcal{R} = \text{Children}(x) \cap \mathcal{S}$ ; // children of  $x$  being repaired
10 if  $\mathcal{R} = \emptyset$  then let  $r = \emptyset$  and  $C = \text{Children}(x)$ ;
11 else // treat all children being repaired as one
12   let  $r = \Theta_{c \in \mathcal{R}} c$  and  $[l_r, u_r] = [\text{false}, \text{true}]$ ;
13   let  $C = \text{Children}(x) \setminus \mathcal{R} \cup \{r\}$ ;
14 let  $\mathcal{F} = \emptyset$ ; // result set of  $(s, f)$  pairs to be computed
15 foreach  $c \in C$  do
16   // Combine bounds from all other children:
17   let  $[l', u'] = [\Theta_{c' \in C \setminus \{c\}} l_{c'}, \Theta_{c' \in C \setminus \{c\}} u_{c'}]$ ;
18   if  $\Theta$  is  $\wedge$  then
19     let  $l_c^* = l^*$ ; let  $u_c^* = u_c \wedge (u^* \vee \neg u')$ ;
20   else //  $\Theta$  is  $\vee$ 
21     let  $l_c^* = l_c \vee (l^* \wedge \neg l')$ ; let  $u_c^* = u^*$ ;
22   if  $c$  is not  $r$  then let  $\mathcal{F} = \mathcal{F} \cup \text{DeriveFixes}(c, \mathcal{S}[c], l_c^*, u_c^*)$ ;
23   else let  $\mathcal{F} = \mathcal{F} \cup \text{DistributeFixes}(\text{MinFix}(l_c^*, u_c^*), C)$ ;
24 return  $\mathcal{F}$ ;

```

in a way such that any repairs on x_2 and x_3 within these bounds ensure that x_1 's target bound is met. Let us focus on setting the target bound for x_2 . As argued above, we would like it to be as loose as possible. Thankfully, because $x_1 \Leftrightarrow x_2 \vee x_3$, x_3 can help "cover" some of x_1 . Specifically, no matter how we end up repairing x_3 , we know it is lower-bounded by $A=C \wedge D < 7$ (denote this formula by l'). Hence, x_3 will certainly cover the $P^* \wedge l'$ part of P^* , leaving x_2 responsible to cover only $P^* \wedge \neg l'$. This observation motivates us to set the lower target bound for x_2 by raising its lower repair bound (denote it by l_c) to $l_c \vee (P^* \wedge \neg l')$ (Line 20) instead of all the way up to $l_c \vee P^*$. On the other hand, x_3 does not help with setting the upper target bound for x_2 . We have to set x_2 's upper target bound to P^* , because if x_2 "overshoots" P^* , $\vee$ -ing it with any x_3 formula will not bring it down. In sum, we set the target bound for x_2 as $[l_c \vee (P^* \wedge \neg l'), P^*] = [P^* \neg (A=C \wedge D < 7), P^*]$. A symmetric argument leads to setting the target bound for x_3 as $[(A=C \wedge D < 7) \vee P^*, P^*]$ (in this case x_2 offers no help to x_3 because it is lower-bounded only by false). The intuition behind pushing the target bound through $\wedge$ is analogous to that described above for $\vee$ but instead boils down to lowering upper bounds as little as possible (as opposed to raising lower bounds). Completing the rest of Example 7, we show the target bounds derived by DeriveFixes for P given repair sites $\{x_4, x_{10}, x_{12}\}$ in Table 1.

Another aspect of DeriveFixes worth mentioning is its handling of the case when multiple repair sites have the same $\wedge$ or $\vee$ parent (which is common because many queries in practice are conjunctive;

Node(s)	target lower bound	target upper bound
$x_1 (P)$	$P^\star$	$P^\star$
x_2	$P^\star \wedge \neg(A=C \wedge D < 7)$	$P^\star$
x_4	$P^\star \wedge \neg(A=C \wedge D < 7)$	$P^\star \vee \neg(D \neq E \vee D > F)$
x_5, x_8, x_9	same as in original predicate	
x_3	$P^\star \vee (A=C \wedge D < 7)$	$P^\star$
x_6	same as in original predicate	
x_7	$P^\star \vee (A=C \wedge D < 7)$	$P^\star \vee \neg(A=C)$
$x_{10} \wedge x_{12}$	$P^\star \wedge \neg(D < 7)$	$P^\star \vee \neg(A=C)$
x_{11}	same as in original predicate	

Table 1. Target lower and upper bounds in Example 5

therefore, their trees have only two levels- the root and the leaves). Since $\wedge$ and $\vee$ are commutative, all such sites can be combined into effectively one site (r in Algorithm 3) to be fixed. In Example 5 above, x_{10} and x_{12} are handled in this manner. Once we obtain a fix for r using MinFix (in conjunctive normal form for $\wedge$ or disjunctive normal form for $\vee$), DistributeFixes distributes the r 's clauses to the repair sites (Line 22) based on syntactic similarities between them.

The following is the main result of this section, which affirms that so long as a candidate set $\mathcal{S}$ of repair sets passes the repair bound check in Section 5.1, there must exist a correct repair for $\mathcal{F}$ and DeriveFixes will find it. This lemma and Lemma 5.3 together imply that our repair bound check is *exact*.

LEMMA 5.4 (EXISTENCE OF CORRECT REPAIR). *Suppose $P^\star \in \text{CreateBounds}(P, \mathcal{S})$. $\text{DeriveFixes}(P, \mathcal{S}, P^\star, P^\star)$ returns $\mathcal{F}$ such that applying $(\mathcal{S}, \mathcal{F})$ to P yields a formula equivalent to $P^\star$.*

In the remainder of this section, we first focus on MinFix, which DeriveFixes uses to find the smallest formula within a target bound. We end with a discussion of complexity, optimality, and, when we cannot guarantee optimality, techniques to mitigate suboptimality.

Finding Smallest Formula with a Bound. Given a target bound $[l^\star, u^\star]$ for a repair site, MinFix needs to find a formula g with the smallest size possible such that $g \in [l^\star, u^\star]$. This goal is intimately related to the *Boolean minimization* problem, which has been well studied and known to be hard [16]. Many practically effective tools have been developed over the years, so our strategy is to leverage these tools for QR-HINT. There are two technical challenges: 1) Boolean minimization is formulated in terms of expressions involving independent Boolean variables, while our formulae involve atomic predicates whose truth values are not independent. 2) Our minimization problem is given a bound as opposed to a single expression that Boolean minimization typically expects.

To address (1), we run a heuristic procedure using Z3 to identify a set $\mathcal{A}$ of “unique” atomic predicates that appear in $l^\star$ and $u^\star$; those that are logically equivalent to others or can be expressed easily in terms of others (e.g., with a negation) are excluded. This procedure does not need to detect or remove intricate dependencies (such that $A > C$ follows from $A > B$ and $C \leq B$); any such dependencies will still be caught later. Then, we map each predicate in $\mathcal{A}$ to a unique Boolean variable and convert $l^\star$ and $u^\star$ into Boolean expressions involving these variables.

To address (2), we note that many practical Boolean minimization tools accept the specification of Boolean expressions as truth tables with possible *don't-care* output entries. Our idea is to use *don't-cares* to encode the constraint implied by the target bound. Specifically, we generate a truth table whose rows correspond to truth assignments of the Boolean variables for $\mathcal{A}$. If a particular assignment is not feasible (which is testable in Z3) due to interacting atomic predicates, we mark the output for the row as *don't-care*. For each feasible assignment, if $l^\star$ and $u^\star$ evaluate to the same truth value, we designate the output for that row to be this value. If $l^\star$ evaluates to false and $u^\star$ evaluates to true, we mark the output as *don't-care*—reflecting the flexibility offered by the bound.

(Note that because $l^* \Rightarrow u^*$, the case where l^* and u^* evaluate to true and false respectively cannot occur.)

The current implementation of QR-HINT uses *ESPRESSO* [14] as the primitive *MinBoolExp* for finding a minimum-size Boolean expression given a truth table with *don't-cares*.

Complexity and Optimality. In our analysis below, let κ denote the combined size of formulae P and P^* . *DeriveFixes*'s main cost comes from calls to *MinFix* and *Z3*. The number of times that *MinFix* is invoked is $|S|$, which is $O(\kappa)$ but is usually a small constant in practice. *MinFix* runs in time exponential in the number of Boolean variables, which is capped at κ . To construct the input truth table for *MinBoolExp*, *MinFix* will also call *Z3* $O(2^\kappa)$ times. Each *Z3* call may take time exponential in the length of its input, though in practice, we time out with an inconclusive answer. Finally, as discussed at the beginning of Section 5, the number of calls to *DeriveFixes* by *RepairWhere* can be worst-case exponential in κ , but in practice it will be $O(\kappa^3)$. Regardless, the overall complexity of *RepairWhere* is exponential in the complexity of the WHERE predicates. Although this worst-case complexity seems daunting, we have found that QR-HINT delivers acceptable performance in practice: thankfully, κ is often small, and the structures of P and P^* and the interdependencies among their atomic predicates tend to be much simpler than, e.g., our Example 5.

The optimality result is presented earlier as Lemma 5.2. Intuitively, the guarantees (which still depend on the primitives *Z3* and *MinBoolExp*) stem from two observations: 1) if repair is limited to a single site, the target bound computed by *DeriveFixes* is indeed the best one can do; and 2) if all sites share the same parent, *DeriveFixes* would effectively process them as a single site. However, target bounds for non-combinable repair sites cannot be set optimally in an independent manner; the approach taken by *DeriveFixes*, which essentially assumes that siblings receive the least amount of help possible from each other when pushing down target bounds, cannot guarantee a minimum-size repair. Indeed, our running example Example 5 with repair sites $\{x_4, x_{10}, x_{12}\}$ is an instance where *DeriveFixes* fails to set target bounds optimally, because x_4 has a different parent from x_{10} and x_{12} . To mitigate this problem, we have developed a more sophisticated algorithm (called *DeriveFixesOPT*) for finding fixes for multiple sites holistically. A full discussion of *DeriveFixesOPT* is beyond the scope of this paper (details in [35]). *DeriveFixesOPT* increases the complexity by another factor of $2^{|S|}$. It is heuristic in nature (as it prioritizes repair sites by how constrained they are) and cannot guarantee optimality beyond Lemma 5.2. However, it does well in practice and better than *DeriveFixes*. Since $|S|$ is small in practice, the complexity overhead is a good price to pay.

EXAMPLE 8. In Example 5, for repair sites $\{x_4, x_{10}, x_{12}\}$, *DeriveFixes* returns fixes $x_4 \mapsto A=B \vee (A=C \wedge D>10) \vee (A=C \wedge D<7)$; $x_{10} \mapsto (A=B \wedge D \neq E) \vee (A=B \wedge D>F)$; $x_{12} \mapsto (A=C \wedge D>10) \vee (A=C \wedge E<5)$. On the other hand, *DeriveFixesOPT* finds the optimal fixes $x_4 \mapsto A=B$; $x_{10} \mapsto D>10$; $x_{12} \mapsto E<5$.

6 GROUP BY STAGE

We check the GROUP BY equivalence assuming Q^* , Q have equivalent FROM and WHERE clauses. We focus on ensuring $\text{FWG}(Q) \equiv \text{FWG}(Q^*)$, regardless of the order and the number of expressions involved in their GROUP BY clauses.

In the following, we consider the case where both Q and Q^* have grouping and/or aggregation. Suppose we have unified the WHERE conditions and GROUP BY expressions in the two queries according to the table mapping $\mathfrak{m}$. Let P denote the resulting formula for Q^* 's WHERE condition (which at this point is logically equivalent to Q 's), and let $\vec{o}$ and $\vec{o}^*$ denote the resulting lists of GROUP BY expressions for Q and Q^* , respectively. Note that the ordering of the GROUP BY expressions is unimportant. Also, if a query involves aggregation but has no GROUP BY, we consider the list of GROUP BY expressions to be an empty list. Same column references across P , $\vec{o}$, and $\vec{o}^*$ are treated as same variables. Our goal is to compute a subset Δ^- of GROUP BY expressions to be removed from Q ,

as well as a set Δ^+ of additional GROUP BY expressions to be added to Q , such that the resulting query will always produce the same grouping of intermediate result tuples (produced by FROM-WHERE) as Q^* . In practice, we may not want to reveal Δ^+ , but instead simply hint that Q misses some GROUP BY expressions. We may repeat the hinting process several times until GROUP BY is completely fixed.

Repairing grouping is trickier than it seems because seemingly very different GROUP BY lists can produce equivalent grouping, as illustrated by the following example.

Example 6.1. Consider two queries over tables $R(A, B)$ and $S(C, D)$:

```
|| SELECT B FROM R, S WHERE B=C GROUP BY B, D; -- Q*
|| SELECT C FROM R, S WHERE B=C GROUP BY C+D, C; -- Q
```

The two queries are equivalent, even though none of the pairs of GROUP BY expressions are equivalent when examined in isolation.

To address this challenge, instead of comparing pairs from $\vec{o}^*$ and $\vec{o}$ in isolation, we holistically consider these lists as well as the WHERE condition, and go back to the definition of GROUP BY as computing a partitioning of intermediate result tuples. Formally, the viability check for this stage is that $\vec{o}$ and $\vec{o}^*$ achieve the same partitioning, or more precisely:

$$V_3 : \text{Check if } \forall t_1, t_2 \in \text{FW}(Q^*) : \bigwedge_i (o_i[t_1] = o_i[t_2]) \Leftrightarrow \bigwedge_i (o_i^*[t_1] = o_i^*[t_2])$$

Here, t_1 and t_2 denote intermediate result tuples, which are known to satisfy P ; we use $o[t]$ to denote evaluating e over t .⁷ This approach underlines our algorithm `FixGrouping` (Algorithm 4).

EXAMPLE 9. Consider the two queries in Example 6.1. The table mapping is trivial and we simply use column names to name variables. We have: P is $B = C$, $\vec{o}^* = [B, D]$, and $\vec{o} = [C + D, C]$. The logical statement that establishes the equivalence of grouping is

$$\begin{aligned} & \forall (A_1, B_1, C_1, D_1), (A_2, B_2, C_2, D_2) : \\ & (B_1 = C_1 \wedge B_2 = C_2) \quad // \text{both } (A_1, B_1, C_1, D_1) \text{ and } (A_2, B_2, C_2, D_2) \text{ satisfy } P \\ & \Rightarrow \left(\begin{array}{l} (B_1 = B_2 \wedge D_1 = D_2) \quad // Q^*'s \text{ grouping criterion} \\ \Leftrightarrow (C_1 + D_1 = C_2 + D_2 \wedge C_1 = C_2) \quad // Q's \text{ grouping criterion} \end{array} \right). \end{aligned}$$

Note that instead of referring to tuples t_1 and t_2 , we simply refer to variables representing their column values in the above.

In `FixGrouping`, to find Δ^- , which are “wrong” expressions in $\vec{o}$, we check, for each o_i , whether it is possible that given $P[t_1] \wedge P[t_2]$, we can have $\bigwedge_i (o_i^*[t_1] = o_i^*[t_2])$ but not $o_i[t_1] = o_i[t_2]$. If yes, that means o_i is wrong with respect to o^* , because while t_1 and t_2 should belong to the same group per o^* , grouping by o_i alone would have forced them into separate groups instead. After identifying all wrong expressions in $\vec{o}$ and removing them, we are left with a partitioning potentially coarser than o^* but otherwise consistent with o^* . We then find Δ^+ to be further added in a similar fashion.

LEMMA 6.2. We say that two lists of GROUP BY expressions are equivalent if they produce the same partitioning for the above query over any database instance. Let $(\Delta^-, \Delta^+) = \text{FixGrouping}(P, \vec{o}, \vec{o}^*)$. Assuming that subroutine `IsSatisfiable` returns no false positives, we have:

Correctness: GROUP BY-stage hint leads to a fixed working query Q_3 that 1) passes the viability check ($\vec{o}, \vec{o}^*$ are equivalent), 2) satisfies $\text{FWG}(Q_3) \equiv \text{FWG}(Q^*)$; and 3) leads to eventual correctness.

Further assuming that `IsSatisfiable` returns no false negatives, we have:

Strong Minimality of Δ^- : Let (Δ_-, Δ_+) denote the minimal Δ^- and Δ^+ respectively, then for any (Δ_-, Δ_+) such that $\vec{o} \setminus \Delta_- \cup \Delta_+$ is equivalent to $\vec{o}^*$, $\Delta^- \subseteq \Delta_-$.

Weak Minimality of Δ^+ : If $\Delta^+ \neq \emptyset$, then there exists no Δ_- such that $\vec{o} \setminus \Delta_-$ is equivalent to $\vec{o}^*$.

⁷Formally, we treat t as an assignment of variables (column references) in e to variables representing corresponding column values in t . Hence, $e[t]$ is an expression obtained from e by replacing each variable (column reference) v with variable $t(v)$.

Algorithm 4: FixGrouping($P, \vec{o}, \vec{o}^*$)**Input** : a formula P and two expression lists $\vec{o}$ and $\vec{o}^*$ **Output** : a pair (Δ^-, Δ^+) , where $\Delta^- \subseteq [1.. \dim(\vec{o})]$ is a subset of indices of $\vec{o}$ and $\Delta^+ \subseteq [1.. \dim(\vec{o}^*)]$ is a subset of indices of $\vec{o}^*$

```

1 let  $\vec{v}$  denote the set of variables in  $P$ ,  $\vec{o}$ , and  $\vec{o}^*$ ;
2 let  $t_1, t_2$  be two assignments of  $\vec{v}$  to new sets of variables  $\vec{v}_1$  and  $\vec{v}_2$ ;
3 let  $G^*$  denote the formula  $\bigwedge_i (o_i^*[t_1] = o_i^*[t_2])$ ;
4 let  $\Delta^- = \emptyset$ ;
5 foreach  $o_i \in \vec{o}$  do
6   if  $\text{IsSatisfiable}(P[t_1] \wedge P[t_2] \wedge G^* \wedge o_i[t_1] \neq o_i[t_2])$  then
7     let  $\Delta^- = \Delta^- \cup \{i\}$ ;
8 let  $G$  denote the formula  $\bigwedge_{i \notin \Delta^-} (o_i[t_1] = o_i[t_2])$ ;
9 let  $\Delta^+ = \emptyset$ ;
10 foreach  $o_i^* \in \vec{o}^*$  do
11   if  $\text{IsSatisfiable}(P[t_1] \wedge P[t_2] \wedge G \wedge o_i^*[t_1] \neq o_i^*[t_2])$  then
12     let  $\Delta^+ = \Delta^+ \cup \{i\}$ ;
13     let  $G = G \wedge o_i^*[t_1] \neq o_i^*[t_2]$ ;
14 return  $(\Delta^-, \Delta^+)$ ;

```

The strong minimality of Δ^- means that we can hint each expression therein as a “must-fix.” The weak minimality of Δ^+ works perfectly as we simply hint that the wrong query needs some additional GROUP BY expressions.

7 HAVING STAGE

At HAVING stage, we aim at further ensuring that $\text{FWGH}(G) \equiv \text{FWGH}(G^*)$ assuming that Q^* and Q unified by a table mapping and have equivalent FROM, WHERE, and GROUP BY. While HAVING can also be modeled as a logical formula, there are new challenges: 1) unlike WHERE, inputs to HAVING formulae are arrays of tuples $[t_1, \dots, t_n]$ instead of single tuples, 2) we need to consider aggregate functions, and 3) we cannot test HAVING alone without considering WHERE’s effect.

EXAMPLE 10. Consider two queries over $R(A, B)$ and $S(C, D)$:

```

|| SELECT A FROM R, S WHERE A=C AND A>4 GROUP BY A, B
||   HAVING A > B + 3 AND 2*SUM(D) > 10; -- Q*
|| SELECT A FROM R, S WHERE A=C GROUP BY A, B, C
||   HAVING C > B + 3 AND SUM(D * 2) > 10 AND A>4; -- Q

```

The two queries are equivalent because $A=C$ in WHERE, because $2*$ distributes over SUM, and because $A>4$ can be either in WHERE or HAVING.

Our strategy is to construct two formulae H^*, H for the HAVING conditions of Q^*, Q respectively, such that equivalence of H^* and H implies $\text{FWGH}(G) \equiv \text{FWGH}(G^*)$. To this end, for each reference to a GROUP BY column in HAVING, we replace it with a variable from the same domain, and we translate HAVING expressions outside aggregate function calls in the same way as we handle WHERE: e.g., $A>B+3$ becomes $A>B+3$. For each reference to a column not in GROUP BY, we introduce an array variable to capture the fact that it refers to a collection of values from rows in the same group. Moreover, for each aggregate function call, we introduce a new array variable to represent the collection of input values if they are computed from an expression, and we use a universally quantified assertion to relate this variable to the source column values: e.g., for $\text{SUM}(D*2)$ we introduce array-valued D_2 to represent $D*2$ values, and we related it to the array-valued D representing D values by asserting

$\forall i \in \mathbb{N} : \mathbf{D}_2[i] = \mathbf{D}[i] \times 2$. Such assertions, along with the WHERE condition and additional inference rules for aggregate functions, go into a context as discussed in Section 3 and illustrated in Example 3.

EXAMPLE 11. For Example 10, HAVING formulae for $Q^\star$, Q are:

$$\begin{array}{ll} (H^\star) & A > B + 3 \wedge (2 \times \text{SUM}(\mathbf{D}) > 10) \\ (H) & C > B + 3 \wedge \text{SUM}(\mathbf{D}_2) > 10 \wedge A > 4 \end{array}$$

We test their equivalence under the following context:

$$C : \left\{ \begin{array}{l} \mathbf{D}, \mathbf{D}_2 \text{ have type } \text{Array}(\mathbb{Z}) \\ A = C \wedge A > 4 \\ \forall i \in \mathbb{N} : \mathbf{D}_2[i] = \mathbf{D}[i] \times 2 \\ \hline \text{SUM has type } \text{Array}(\mathbb{Z}) \rightarrow \mathbb{Z} \\ \forall c \in \mathbb{Z}, \mathbf{X} \text{ and } \mathbf{Y} \text{ of type } \text{Array}(\mathbb{Z}) : \\ (\forall i \in \mathbb{N} : \mathbf{X}[i] \times c = \mathbf{Y}[i]) \Rightarrow \text{SUM}(\mathbf{X}) \times c = \text{SUM}(\mathbf{Y}) \end{array} \right\},$$

In the above, the assertions underneath the horizontal line are generic assertions encoding properties of aggregate functions useful for inferring equivalences. Only those relevant to Example 10 are listed here; for a complete list see [35].

The viability check for HAVING (Theorem 3.1, stage 4) is that H is logically equivalent to $H^\star$ under HAVING base context C , i.e.:

$$V_4 : \text{Check if } H \Leftrightarrow H^\star \text{ under } C$$

Note that this check implicitly applies to all groups. If a constraint solver fails to establish equivalence, we invoke the exact same procedures as for WHERE to find a repair.

LEMMA 7.1. HAVING-stage hint leads to a fixed working query Q_4 with HAVING condition that 1) passes the viability check; 2) satisfies $\text{FWGH}(Q_4) \equiv \text{FWGH}(Q^\star)$; and 3) leads to eventual correctness.

As with WHERE, the correctness of the above lemma relies only on the fact that Z3 inference is sound with respect to the logic exercised by H , $H^\star$, and C and that MinBoolExp always finds a Boolean formula equivalent to its given input. We could additionally guarantee optimality similar to Lemma 5.2 by making the same assumptions therein (completeness of Z3 inference and optimality of MinBoolExp) plus the additional assumption that the context C encodes all properties of aggregate functions relevant to inference.

8 SELECT STAGE

This stage aims at fixing SELECT as needed to ensure $Q \equiv Q^\star$, assuming that they already have equivalent FROM, WHERE, GROUP BY and HAVING. We test the equivalence between SELECT expressions with a context C dependent on the type of the query: if the queries are SPJ, we simply assert the WHERE condition in C ; if the queries are SPJA, we use the same C defined by the HAVING-stage.

Let $\vec{o}$ and $\vec{o}^\star$ denote the resulting ordered lists of SELECT expressions for Q , $Q^\star$, respectively. The viability check (V_5) is that $\dim(\vec{o}) = \dim(\vec{o}^\star)$ and $\vec{o}[i]$ is equivalent to $\vec{o}^\star[i]$ for $1 \leq i \leq \dim(\vec{o}^\star)$, i.e. both SELECTs have the same number of expressions and expressions on the same index position are equivalent. If SELECT clauses are not equivalent between $Q^\star$, Q , our goal becomes to compute Δ^- of SELECT expression to be removed from Q at the corresponding index position and Δ^+ of expressions to be added to Q at the corresponding index position.

The algorithm checks the equivalence between $(\vec{o}[i], \vec{o}^\star[i])$ and add Δ^- and Δ^+ respectively if they are inequivalent. Finally, excessive expressions in Q or $Q^\star$ will also be added to Δ^- and Δ^+ respectively. After fixing SELECT, we guarantee $Q^\star \equiv Q$.

9 EXPERIMENTS

We test three aspects of QR-HINT: coverage, accuracy, and running time. For coverage, we test the ability of QR-HINT to fix wrong queries that arise in real-world classroom settings. For accuracy and running time, we focus on Algorithm 1, which is the bottleneck of QR-HINT due to calls to `DeriveFixes` or `DeriveFixesOPT`. As fix minimization incurs exponential time, we examine 1) how the number of unique predicates affects running time, 2) how close the generated repairs are to the optimal if queries are not conjunctive, 3) a comparison between the running time and optimality of `DeriveFixes` and `DeriveFixesOPT`. In general, `DeriveFixesOPT` strives for smaller fixes and hence incurs longer running time than `DeriveFixes`.

Implementation/Test Environment. We implemented QR-HINT in Python 3.10 using Apache Calcite [6] to parse SQL queries and Z3 SMT Solver [24] to test constraint satisfiability. We used *ESPRESSO* in PyEDA [27] for fix minimization. We ran the experiments locally on a 64-bit Ubuntu 20.04 LTS server with 3.20GHz Intel Core i7-8700 CPU and 32GB 2666MHz DDR4.

Test Data Preparation. To prepare the first test dataset, denoted *STUDENTS*, we examined 2,000+ real student queries from an undergraduate database course in one semester at the first author's institution. These queries came from 4 introductory-level SQL questions (with 4 reference queries), and altogether they included 341 wrong queries. Out of these, 35 (11%) used SQL features not supported by QR-HINT (see limitations at the end of Section 3). Hence, we end up with 306 supported wrong queries in *STUDENTS*. (At the time of writing, we are still exploring with the institutional review board the possibility of making this dataset publicly available.)

To further expand coverage of errors, we cross-checked *STUDENTS* queries with the list of SQL issues indicative of semantic errors categorized by Brass et al. [12] (which did not publish a query dataset). Out of the 43 issues in [12], 18 involve SQL features not currently supported by QR-HINT, but they only make up for a small minority (11.4%) of the observed instances as reported by [12]. Out of the 25 issues QR-HINT should support, 17 are already represented in the 306 *STUDENTS* queries. To cover the remaining 8, we handcrafted two queries according to each issue and added to the dataset; we also handcrafted corresponding reference queries (free from any issue in [12]). We denote the resulting dataset *STUDENTS+*, with 322 queries having errors/issues.

Our second test dataset, denoted *TPCH*, is based on TPC-H [7] schema and queries, with synthetic errors injected. This dataset allows us to stress-test QR-HINT with queries that are more complex than *STUDENTS*. Also, because errors are synthetic, we have the “ground-truth” repair sites and fixes, allowing us to easily assess the optimality of QR-HINT fixes. Most *WHERE* conditions in TPC-H queries are conjunctive: we chose 7 TPC-H queries with conjunctions of 4,5,6,7,9,10,11 atomic predicates (TPC-H Query 4,3,10,9,5,8,21 respectively). Since we did not find a TPC-H query with exactly 8 predicates, we synthesized one by removing one predicate from TPC-H Query 5. For each query, we then introduced errors into two atomic predicates to make the wrong query, which remained conjunctive. Thus, each pair of wrong and reference queries has 6-13 unique atomic predicates. Furthermore, to test cases beyond conjunctive *WHERE* conditions, we chose TPC-H Query 7, whose *WHERE* contains multiple nested *AND* and *OR*, and created 5 wrong queries by injecting 1-5 errors by changing atomic predicates or logical operators. For fair comparison, we ensured that the number of unique atomic predicates is always 10 between the reference query and each wrong query.

9.1 Results and Discussion

STUDENT+. To test coverage and optimality of QR-HINT, we ran QR-HINT for the 322 *STUDENT+* queries with errors/issues, along with their reference queries, and examined all QR-HINT fixes. For the 25 issues in [12] that QR-HINT should support, we found that they were handled in three

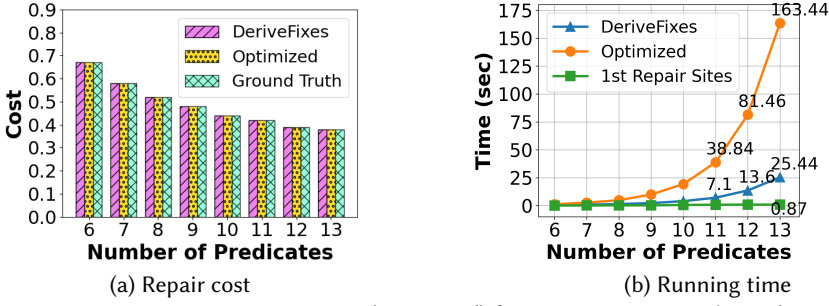


Fig. 2. DeriveFixes vs. DeriveFixesOPT (Optimized) for conjunctive WHERE (TPCH)

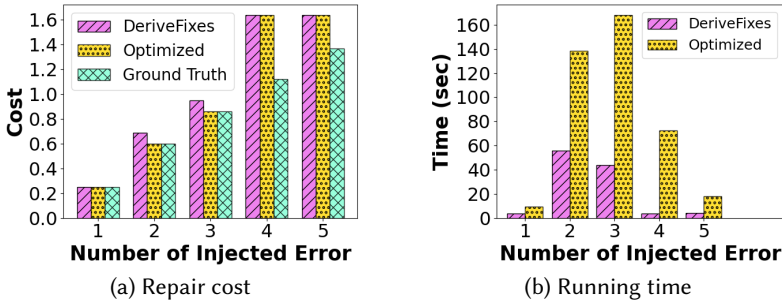


Fig. 3. DeriveFixes vs. DeriveFixesOPT (Optimized) for nested AND/OR (TPCH)

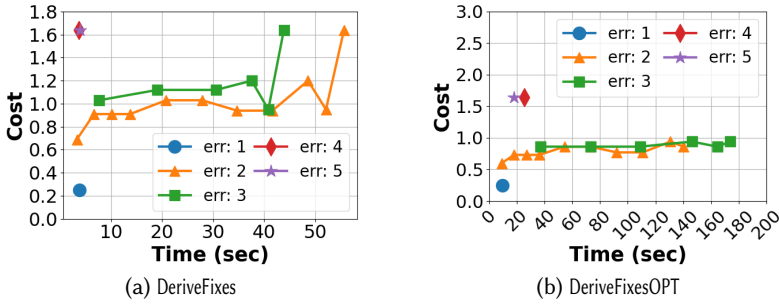


Fig. 4. Cost of repairs found during course of execution

ways: 1) 11 of them were indeed errors, and QR-HINT correctly identified and fixed them all; 2) 3 of them were efficiency/stylistic issues where the queries were semantically still correct (e.g., logically correct WHERE containing some tautological conditions, such as $A \geq B \text{ OR } A < B$), and QR-HINT did not flag any error; 3) the remaining 11 of them were also efficiency/stylistic issues (e.g., unnecessarily joining a primary key with its corresponding foreign key but only projecting the foreign key column), but QR-HINT failed to detect query equivalence in this case and suggested some fixes. This last category is the only case where QR-HINT showed suboptimal behavior, though its suggested fixes still lead to correct queries, and with the interesting side effect of resolving efficiency/stylistic issues. The detailed analysis can be found in [35]. It is worth noting that QR-HINT perfectly handles all of the 10 most common issues in [12].

QR-HINT's average running time per query on STUDENT+ is 0.2 seconds, using DeriveFixes. However, note that most STUDENT+ queries are rather simple, with conjunctive WHERE (which does not need DeriveFixesOPT for optimality) and at most 5 unique atomic predicates. Therefore, we further stress-tested QR-HINT using TPCB.

TPCH, conjunctive WHERE with varying number of atomic predicates. Here, we study QR-HINT's running time and optimality (as measured by repair cost, the lower the best) as we vary the number of atomic predicates involved in repairing WHERE. We compare versions of QR-HINT using *DeriveFixes* vs. *DeriveFixesOPT*, both set to explore up to two repair sites. Figure 2a confirms that for conjunctive queries, both always return optimal repairs according to the ground truth, regardless of the size of WHERE. (Note that the repair cost is not proportional to the number of atomic predicates because it is normalized by the query sizes per Equation (1)). Figure 2b shows that as expected, both have running times exponential in the number of unique atomic predicates, but *DeriveFixes* runs much faster than *DeriveFixesOPT*. Furthermore, the plot labeled "1st Repair Sites" shows that it takes less than one second for QR-HINT to find the first *viable* (not necessarily optimal) repair site, so there is additional room to trade optimality for faster running time.

TPCH, WHERE with nested AND/OR and varying number of injected errors. As shown in Figure 3a, when the optimal repair (according to the ground truth) involves only one repair site (a single error), both *DeriveFixes* and *DeriveFixesOPT* are able to find this optimal repair, confirming Lemma 5.2. When there are more errors (2-3), *DeriveFixes* returns suboptimal repairs while *DeriveFixesOPT* is still able to find optimal or near-optimal repairs (for the cases of 2 and 3 errors, respectively). However, with 4-5 errors—which are arguably not the cases QR-HINT targets—both suffer from suboptimality because they are set to explore up to two repair sites; in fact, both decided that it was best to just repair the whole WHERE condition. Figure 3b shows that *DeriveFixesOPT*'s better optimality comes at the expense of slower speed than *DeriveFixes*, however. Interestingly, with 4-5 errors, both run faster than with 2-3 errors, because the large numbers of errors severely limit the number of possibilities of single- and 2-site repairs, speaking to the effectiveness of *CreateBounds* in quickly spotting and bailing out of difficult situations.

Finally, Figure 4 shows all unpruned viable repairs found during QR-HINT's course of execution, in terms of when they were found and how much they cost; there is one trace for each execution. Traces for 1 (blue), 4 (red), and 5 (purple) errors degenerate into single dots because QR-HINT eventually finds only one solution as viable repair options are limited. Recall that we heuristically prioritize the viable repairs to consider, but there is no guarantee that a cheaper repair will always be found earlier. Hence, there are fluctuations in the repair costs over time, although the general trends are up, confirming the effectiveness of our heuristic. Furthermore, note that the lowest-cost repairs tend to surface early during execution. In closing, while the total and worst-case running times of QR-HINT grow exponentially in query size, in practice the running times are reasonable considering that QR-HINT is intended for education settings, where returning hints instantaneously may not be necessary or desirable for learning. With the observation that QR-HINT often returns some low-cost repairs early, we can offer them as preliminary hints to get students thinking, while QR-HINT continues to look for better repairs in the meantime.

10 USER STUDY

We conducted a small-scale user study to evaluate QR-HINT: 1) whether students can understand what is wrong with the suggested hints, and 2) how the hints generated by QR-HINT compare with ones provided by "expert users" (teaching assistants in our study).

Participants. We recruited 38 students who have taken/are taking a graduate or undergraduate database course. Except for an incentive of receiving a small gift card and practicing SQL, the participation was voluntary. In the end, we collected 15 complete and valid answers. A possible explanation for the low completion rate was the significant effort required to debug SQL queries with subtle mistakes (we observed that some participants took more than an hour to finish). We considered the possibility of recruiting participants from other sources (e.g., Amazon Mechanical Turk), but decided against it because they would not represent our targeted population (students).

Furthermore, given the significant effort required from the participants as observed above, it would be hard to incentivize participants who are not actively learning SQL: a low reward would turn them away, while a high reward might encourage undesirable behaviors.

Preparation. To design the survey, we first performed an analysis of the STUDENTS queries to get a sense of what the common errors were. Overall, most errors came from WHERE and HAVING (130 out of 341 are wrong due to WHERE); students often missed join conditions for queries involving many tables. Other common errors include incorrect/redundant/missing tables in FROM, incorrect order and missing/redundant expressions in SELECT, and incorrect expressions in GROUP BY. We decided not to use the same queries from STUDENTS, as our participants had done the same/similar homework previously, which might bias the results. Nonetheless, based on these observations, we designed four SQL questions using a different schema, DBLP (details in [35]). For each question, we crafted a wrong solution containing one or more mistakes: two WHERE errors for Q_1 , one GROUP BY error and one SELECT error for Q_2 , one WHERE error for Q_3 , and one each WHERE and HAVING errors in Q_4 . Even though the queries are over a different schema, the errors above faithfully reflect real errors from STUDENTS, and they are consistent with the common errors found by others [2, 12].

Then, we performed a small study with four graduate teaching assistants (TAs) to generate hints for these queries. Each TA was asked to pinpoint all mistakes in each query and offer hints, as if they were helping students debug wrong queries. To simulate an office-hour setting, we asked TAs to finish all four questions in one sitting, with no help from QR-HINT. We collected all hints provided by the TAs as “expert” hints.

Next, we ran QR-HINT on all wrong queries to obtain repair sites and fixes. We removed fixes and only showed repair sites to the participants as hints. To prevent participants from recognizing the source of hints (experts vs. QR-HINT) by their wording, we paraphrased all hints to use a common template “In [SQL clause], [hint]” and standard wording.

Tasks. Using the four queries, each participant saw and completed three questions. Students were required to complete questions on Q_1 and Q_2 , and they completed one of Q_3 and Q_4 at random. For each question, students were given the database schema, problem statement in English, and the wrong SQL query, and were asked to explain what is wrong with the query. For creating *treatment* and *control* groups, students received hints from QR-HINT for either Q_1 or Q_2 (not both) at random, and for the other one they were asked to detect errors without any hints provided; the order of the two questions with and without hints was also chosen at random. For the last question, participants received Q_3 or Q_4 at random, and we showed the union of hints (mixed together) generated by the TAs as well as by QR-HINT, and asked participants to categorize each hint as one of the following: “Unhelpful or incorrect”, “Helpful but require thinking”, and “Obvious and giving away the answer”. Participants were asked to finish all questions in one sitting. We recorded the time a participant spent on each question⁸. In our study, for Q_1 , 8 students answered it with no hints and 7 with hints from QR-HINT. For Q_2 , these numbers are 7 and 8 respectively. For the third question, 7 received Q_3 and 8 received Q_4 .

Result and Analysis. Our results for Q_1 and Q_2 show that participants were better at identifying at least one error in the query given the hints provided by QR-HINT compared to no hints. As shown in Figure 5a and Figure 5b, 100% and 87.3% of the participants were able to identify at least one of the two errors in the wrong query in Q_1 and Q_2 respectively after receiving hints from QR-HINT, as opposed to 14.3% and 71.4% who were able to do so without a hint. While there is a single participant who correctly identified both errors without any hint for Q_1 , this participant

⁸ Q_1 without/with hints took 704s/460s on average; Q_2 took 756s/658s. Students completed the survey asynchronously, so the time recorded may not be accurate.

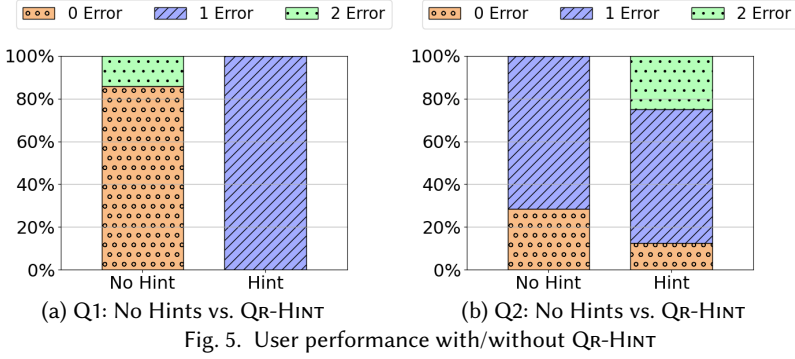


Fig. 5. User performance with/without QR-HINT

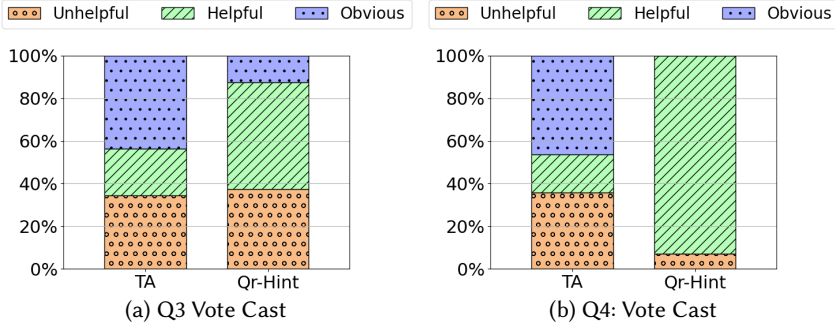


Fig. 6. Hint categorization from participants for Q3, Q4

spent more than 20 minutes doing so, while most participants spent no more than 10 minutes on the same question without hints.

Q3 and Q4 are used to evaluate whether QR-HINT provided hints that are comparable to the ones given by teaching assistants in terms of their quality. For Q3, there are four TA hints and one hint from QR-HINT; and there are four TA hints and two hints generated by QR-HINT for Q4. For all responses, we sum up the number of times participants vote for each of the three categories of hint ranks: “Obvious”, “Unhelpful”, and “Helpful”. The results are shown in Figures 6a, 6b. In summary, the quality of TAs’ hints varies greatly as perceived by participants. On the other hand, QR-HINT is consistently perceived by participants as “helpful but require thinking”, which might be best suited for classroom settings.

11 CONCLUSION AND FUTURE WORK

We presented QR-HINT, a framework for automatically generating hints and suggestions for fixes for a wrong SQL query with respect to a reference query. We developed techniques to fix all clauses in a query and gave theoretical guarantees. There are multiple intriguing directions of future work, including the support of more complex constructs such as subqueries, outer-joins (NULL), and database constraints. In addition, developing techniques to overcome the limitations of the SMT solver and improve the system’s scalability is also an important next step. In the meantime, we are implementing a graphical user interface so that QR-HINT can better assist students/TAs in database courses. With that, we can then conduct a larger-scale user study to further understand the effectiveness of QR-HINT and better prepare for scaled deployment in large database courses.

ACKNOWLEDGMENTS

This work is supported by NSF award IIS-2008107.

REFERENCES

- [1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of databases: the logical level*. Addison-Wesley Longman Publishing Co., Inc.
- [2] Alireza Ahadi, Julia Prior, Vahid Behbood, and Raymond Lister. 2016. Students' semantic mistakes in writing seven different types of SQL queries. In *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*. 272–277.
- [3] Umair Z. Ahmed, Zhiyu Fan, Jooyong Yi, Omar I. Al-Bataineh, and Abhik Roychoudhury. 2022. Verifix: Verified Repair of Programming Assignments. 31, 4, Article 74 (jul 2022), 31 pages.
- [4] Alfred V. Aho, Yehoshua Sagiv, and Jeffrey D. Ullman. 1979. Equivalences among relational expressions. *SIAM J. Comput.* 8, 2 (1979), 218–246.
- [5] Marcelo Arenas, Pablo Barceló, Leonid Libkin, Wim Martens, and Andreas Pieris. 2022. *Database Theory*. Open source at <https://github.com/pdm-book/community>.
- [6] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J Mior, and Daniel Lemire. 2018. Apache calcite: A foundational framework for optimized query processing over heterogeneous data sources. In *Proceedings of the 2018 International Conference on Management of Data*. 221–230.
- [7] TPC Benchmark. [n. d.]. <http://www.tpc.org/tpch>.
- [8] Berkay Berabi, Jingxuan He, Veselin Raychev, and Martin Vechev. 2021. TFix: Learning to Fix Coding Errors with a Text-to-Text Transformer. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 780–791. <https://proceedings.mlr.press/v139/berabi21a.html>
- [9] Sahil Bhatia, Pushmeet Kohli, and Rishabh Singh. 2018. Neuro-symbolic program corrector for introductory programming assignments. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. IEEE, 60–70.
- [10] Stefan Brass and Christian Goldberg. 2004. Detecting Logical Errors in SQL Queries.. In *Grundlagen von Datenbanken*. 28–32.
- [11] Stefan Brass and Christian Goldberg. 2005. Proving the safety of SQL queries. In *Fifth International Conference on Quality Software (QSIC'05)*. IEEE, 197–204.
- [12] Stefan Brass and Christian Goldberg. 2006. Semantic errors in SQL queries: A quite complete list. *Journal of Systems and Software* 79, 5 (2006), 630–644. <https://doi.org/10.1016/j.jss.2005.06.028> Quality Software.
- [13] Stefan Brass, Christian Goldberg, and Alexander Hinneburg. 2003. *Detecting semantic errors in SQL queries*. Technical Report. Technical Report, University of Halle. https://dbs.informatik.uni-halle.de/sqlint/semerr_techrep.pdf
- [14] Robert K Brayton, Gary D Hachtel, Lane A Hemachandra, A Richard Newton, and Alberto Luigi M Sangiovanni-Vincentelli. 1982. A comparison of logic minimization strategies using ESPRESSO: An APL program package for partitioned logic minimization. In *Proceedings of the International Symposium on Circuits and Systems*. 42–48.
- [15] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [16] David Buchfuhrer and Christopher Umans. 2011. The complexity of Boolean formula minimization. *J. Comput. Syst. Sci.* 77, 1 (2011), 142–153. <https://doi.org/10.1016/j.jcss.2010.06.011>
- [17] Ashok K. Chandra and Philip M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. In *Proceedings of the Ninth Annual ACM Symposium on Theory of Computing (STOC '77)*. Association for Computing Machinery, 77–90.
- [18] Bikash Chandra, Ananyo Banerjee, Udbhas Hazra, Mathew Joseph, and S Sudarshan. 2021. Edit Based Grading of SQL Queries. In *8th ACM IKDD CODS and 26th COMAD*. 56–64.
- [19] Bikash Chandra, Bhupesh Chawda, Biplab Kar, KV Reddy, Shetal Shah, and S Sudarshan. 2015. Data generation for testing and grading SQL queries. *The VLDB Journal* 24, 6 (2015), 731–755.
- [20] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. Sequencer: Sequence-to-sequence learning for end-to-end program repair. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1943–1959.
- [21] Shumo Chu, Brendan Murphy, Jared Roesch, Alvin Cheung, and Dan Suciu. 2018. Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries. *Proc. VLDB Endow.* 11, 11 (jul 2018), 1482–1495.
- [22] Shumo Chu, Chenglong Wang, Konstantin Weitz, and Alvin Cheung. 2017. Cosette: An Automated Prover for SQL.. In *CIDR*.
- [23] Shumo Chu, Konstantin Weitz, Alvin Cheung, and Dan Suciu. 2017. HoTTSQL: Proving query rewrites with univalent SQL semantics. *ACM SIGPLAN Notices* 52, 6 (2017), 510–524.
- [24] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. 337–340.

- [25] Benjamin Dietrich and Torsten Grust. 2015. A SQL Debugger Built from Spare Parts: Turning a SQL: 1999 Database System into Its Own Debugger. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* (Melbourne, Victoria, Australia) (SIGMOD '15). Association for Computing Machinery, New York, NY, USA, 865–870. <https://doi.org/10.1145/2723372.2735358>
- [26] Haoran Ding, Zhaoguo Wang, Yicun Yang, Dexin Zhang, Zhenglin Xu, Haibo Chen, Ruzica Piskac, and Jinyang Li. 2023. Proving Query Equivalence Using Linear Integer Arithmetic. *Proceedings of the ACM on Management of Data* 1, 4 (2023), 1–26.
- [27] Chris Drake. [n. d.]. *Python EDA*. <https://pyeda.readthedocs.io/>
- [28] Curtis Fenner. 2019. <https://cs.stackexchange.com/questions/110674/is-query-equivalence-decidable>.
- [29] Amir Gilad, Zhengjie Miao, Sudeepa Roy, and Jun Yang. 2022. Understanding Queries by Conditional Instances. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*. Association for Computing Machinery, 355–368.
- [30] Christian Goldberg. 2009. Do you know SQL? About semantic errors in database queries. In *7th Workshop on Teaching, Learning and Assessment in Databases, Birmingham, UK, HEA*. Citeseer.
- [31] Torsten Grust and Jan Rittinger. 2013. Observing SQL Queries in Their Natural Habitat. *ACM Trans. Database Syst.* 38, 1, Article 3 (apr 2013), 33 pages. <https://doi.org/10.1145/2445583.2445586>
- [32] Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. Association for Computing Machinery, 465–480.
- [33] Rahul Gupta, Aditya Kanade, and Shirish Shevade. 2019. Deep reinforcement learning for syntactic error repair in student programs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 930–937.
- [34] Rahul Gupta, Aditya Kanade, and Shirish Shevade. 2019. Neural attribution for semantic bug-localization in student programs. *Advances in Neural Information Processing Systems* 32 (2019).
- [35] Yihao Hu, Amir Gilad, Kristin Stephens-Martinez, Sudeepa Roy, and Jun Yang. 2024. Qr-Hint: Actionable Hints Towards Correcting Wrong SQL Queries. *arXiv preprint arXiv:2404.04352* (2024).
- [36] Jinru Hua, Mengshi Zhang, Kaiyuan Wang, and Sarfraz Khurshid. 2018. Towards practical program repair with on-demand candidate generation. In *Proceedings of the 40th international conference on software engineering*. 12–23.
- [37] Tomasz Imieliński and Witold Lipski Jr. 1989. Incomplete information in relational databases. In *Readings in Artificial Intelligence and Databases*. Elsevier, 342–360.
- [38] Yannis E Ioannidis and Raghu Ramakrishnan. 1995. Containment of conjunctive queries: Beyond relations as sets. *ACM Transactions on Database Systems (TODS)* 20, 3 (1995), 288–324.
- [39] T. S. Jayram, Phokion G. Kolaitis, and Erik Vee. 2006. The Containment Problem for Real Conjunctive Queries with Inequalities. In *Proceedings of the Twenty-Fifth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS '06)*. Association for Computing Machinery, 80–89.
- [40] Anthony Klug. 1988. On conjunctive queries containing inequalities. *Journal of the ACM (JACM)* 35, 1 (1988), 146–160.
- [41] Jarosław Kwiecień, Jerzy Marcinkowski, and Piotr Ostropolski-Nalewaja. 2022. Determinacy of Real Conjunctive Queries. The Boolean Case. In *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. Association for Computing Machinery, 347–358. <https://doi.org/10.1145/3517804.3524168>
- [42] Aristotelis Leventidis, Jiahui Zhang, Cody Dunne, Wolfgang Gatterbauer, HV Jagadish, and Mirek Riedewald. 2020. QueryVis: Logic-based diagrams help users understand complicated SQL queries faster. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2303–2318.
- [43] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. Coconut: combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*. 101–114.
- [44] Zhengjie Miao, Sudeepa Roy, and Jun Yang. 2019. Explaining wrong queries using small examples. In *Proceedings of the 2019 International Conference on Management of Data*. 503–520.
- [45] Daniel Perelman, Sumit Gulwani, and Dan Grossman. 2014. Test-driven synthesis for automated feedback for introductory computer science assignments. *Proceedings of Data Mining for Educational Assessment and Feedback (ASSESS 2014)* (2014).
- [46] Chris Piech, Jonathan Huang, Andy Nguyen, Mike Phulsuksombati, Mehran Sahami, and Leonidas Guibas. 2015. Learning program embeddings to propagate feedback on student code. In *International conference on machine Learning*. PMLR, 1093–1102.
- [47] Kai Presler-Marshall, Sarah Heckman, and Kathryn Stolee. 2021. SQLRepair: identifying and repairing mistakes in student-authored SQL queries. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE, 199–210.
- [48] Kelly Rivers and Kenneth R Koedinger. 2017. Data-driven hint generation in vast solution spaces: a self-improving python programming tutor. *International Journal of Artificial Intelligence in Education* 27 (2017), 37–64.

- [49] Yehoshua Sagiv and Mihalis Yannakakis. 1980. Equivalences among relational expressions with the union and difference operators. *Journal of the ACM (JACM)* 27, 4 (1980), 633–655.
- [50] Oded Shmueli. 1993. Equivalence of Datalog Queries is Undecidable. *J. Log. Program.* 15, 3 (Feb. 1993), 231–241.
- [51] Rishabh Singh, Sumit Gulwani, and Armando Solar-Lezama. 2013. Automated Feedback Generation for Introductory Programming Assignments. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*. Association for Computing Machinery, 15–26.
- [52] Boris Trahtenbrot. 1950. The impossibility of an algorithm for the decision problem for finite domains. In *Doklady Akademii Nauk SSSR*, Vol. 70. 569–572.
- [53] Margus Veanes, Nikolai Tillmann, and Jonathan de Halleux. 2010. Qex: Symbolic SQL Query Explorer. In *Proceedings of the 16th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning (Dakar, Senegal) (LPAR'10)*. Springer-Verlag, Berlin, Heidelberg, 425–446.
- [54] Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. Association for Computing Machinery, 481–495.
- [55] Zhaoguo Wang, Zhou Zhou, Yicun Yang, Haoran Ding, Gansen Hu, Ding Ding, Chuzhe Tang, Haibo Chen, and Jinyang Li. 2022. WeTune: Automatic Discovery and Verification of Query Rewrite Rules (*SIGMOD '22*). Association for Computing Machinery, 94–107.
- [56] Yingfei Xiong, Xinyuan Liu, Muhan Zeng, Lu Zhang, and Gang Huang. 2018. Identifying patch correctness in test-based program repair. In *Proceedings of the 40th international conference on software engineering*. 789–799.
- [57] Qi Zhou, Joy Arulraj, Shamkant Navathe, William Harris, and Dong Xu. 2019. Automated verification of query equivalence using satisfiability modulo theories. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1276–1288.

Received October 2023; revised January 2024; accepted March 2024